Decoding Nonbinary LDPC Codes via Proximal-ADMM Approach

Yongchao Wang, *Senior Member, IEEE*, and Jing Bai

***Abstract*— In this paper, we focus on decoding nonbinary low-density parity-check (LDPC) codes in Galois fields of characteristic two via the proximal alternating direction method of multipliers (proximal-ADMM). By exploiting Flanagan/Constant-Weighting embedding techniques and the decomposition technique based on three-variables parity-check equations, two efficient proximal-ADMM decoders for nonbinary LDPC codes are proposed. We show that both of them are theoretically guaranteed convergent to some stationary point of the decoding model and either of their computational complexities in each proximal-ADMM iteration scales linearly with LDPC code's length and the size of the considered Galois field. Moreover, the decoder based on the Constant-Weight embedding technique satisfies the favorable property of codeword symmetry. Simulation results demonstrate their effectiveness in comparison with state-of-the-art LDPC decoders.**

***Index Terms*— Nonbinary low-density parity-check (LDPC) codes, Galois fields of characteristic two, proximal alternating direction method of multipliers (proximal-ADMM), quadratic programming (QP).**

I. Introduction

NONBINARY low-density parity-check (LDPC) codes [1] in Galois fields of characteristic 2 ($\mathbb{F}_{2^q}$) are favorable in high-data-rate communication systems and storage systems [2] since they possess many desirable merits from the viewpoints of practical applications. For example, nonbinary LDPC codes have greater ability to eliminate short cycles (especially 4-cycles) and display better error-correction performance [3]. Moreover, nonbinary LDPC codes have good ability to resist burst errors by combining multiple burst bit errors into fewer nonbinary symbol errors [4]. Furthermore, a nonbinary LDPC code can provide a higher data transmission rate and spectral efficiency when it is combined with a high-order modulation scheme [5].

Typical decoding algorithms for nonbinary LDPC codes, such as the sum-product [1] [6], are based on the belief propagation (BP) strategy. In [1], Davey and Mackay first investigated nonbinary LDPC codes and the corresponding nonbinary BP algorithm. Later, Declercq and Fossorier in [6] optimized nonbinary BP algorithms by reducing the computational complexity of check-node processing. However, nonbinary BP-like decoding algorithms are heuristic from a theoretical viewpoint since their convergence performance cannot be guaranteed in theory. Meanwhile, analyzing the behavior of nonbinary BP-like decoding algorithms is often difficult and the corresponding results are very limited [7].

In recent years, mathematical programming (MP) techniques, such as linear programming (LP) and quadratic programming (QP) [8], are applied to decoding LDPC codes. These decoding techniques have attracted significant attention from researchers in the error-correction coding/decoding field due to their analyzable decoding performance, such as convergence and codeword symmetrical property. The first MP decoding technique was proposed by Feldman *et al.* [11], who relaxed the maximum-likelihood (ML) decoding problem to a linear program for binary LDPC codes. In comparison with classical BP decoders, two issues must be considered: one is computational complexity of the general LP solving algorithms, such as the interior point method [15] and simplex method [16], which are prohibitive in practical applications and the other is its inferior error-correction performance in low SNR regions [11]. For the first issue, several improved MP decoders for binary LDPC codes are proposed. In [17], Barman *et al.* applied the alternating direction method of the multipliers (ADMM) technique [18] to solve the original LP decoding problem [11]. Comparable with existing LP decoders, the decoding complexity of the ADMM-based decoders is greatly reduced. However, it is still expensive from a practical viewpoint since it involves costly projection operations onto parity polytopes in each iteration. Later, Zhang and Siegel in [19] optimized the parity polytope projection algorithm based on the cut search algorithm proposed in [28]. In [20], the parity polytope projection was further simplified to an efficient simplex projection algorithm. In [21], Jiao *et al.* proposed a cheap projection algorithm, which can be implemented. Specifically, this projection algorithm employed a cut-searching method in [19] to identify which facet to project onto and then performed the projection operation via the simplex method [20]. In [22], Wei and Banihashemi proposed an iterative check-polytope projection algorithm to reduce the complexity of the LP decoding algorithm. Moreover, a projection reduction technique was investigated in [23] to reduce the number of Euclidean projections onto

Manuscript received 13 January 2020; revised 6 December 2021; accepted 21 January 2022. Date of publication 31 January 2022; date of current version 18 August 2022. The work of Yongchao Wang and Jing Bai was supported in part by the National Science Foundation of China under Grant 61771356. *(Corresponding authors: Jing Bai; Yongchao Wang.)*

Yongchao Wang is with the State Key Laboratory of Integrated Services Networks, Xidian University, Xi'an 710071, China (e-mail: ychwang@mail.xidian.edu.cn).

Jing Bai is with the School of Information Science and Technology, Shijiazhuang Tiedao University, Shijiazhuang 050043, China (e-mail: jingbai01@126.com).

Communicated by A. Thangaraj, Associate Editor for Coding and Decoding.

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TIT.2022.3147906.

Digital Object Identifier 10.1109/TIT.2022.3147906

the check polytope. Bai *et al.* in [24] proposed an efficient ADMM-based LP decoding algorithm via the three-variables parity-check equations decomposition technique. For the second issue of error-correction performance, Taagavi and Siegel in [25] designed an adaptive LP decoder to improve LP decoding performance by adaptively adding necessary parity-check constraints. Later, different cut-generating algorithms were designed in [26]–[28] to eliminate unexpected pseudo-codewords and improve the error-correction performance of LP decoding. Rosnes and Helmling in [29] considered adaptive linear programming decoding of linear codes over prime fields and efficient separation of the underlying inequalities describing the decoding polytope which was done through dynamic programming. In addition, Liu and Draper in [30] and Bai *et al.* in [10] proposed improved ADMM-based penalized decoding algorithms to enhance the error-correction performance of LP decoding in low SNR regions.

In comparison with MP decoding techniques for binary LDPC codes, the corresponding approaches for nonbinary cases are limited. Of particular relevance is the work [9] where LP decoding was first generalized to nonbinary LDPC codes. However, nonbinary LP decoding encounters a similar computational complexity problem as the binary case when using general LP solvers. To overcome this problem, Goldin and Burshtein in [31] and Punekar *et al.* in [32] applied the coordinate ascent method to solve the dual problems of the original LP decoding problem in [33] and [34]. Punekar and Flanagan in [35] adopted the binary LP decoding idea in [36] and proposed a trellis-based algorithm for check node processing to reduce the complexity of nonbinary LP decoding. In addition, another nonbinary LP decoding scheme was introduced in [37] by using constant-weight binary vectors to represent elements in $\mathbb{F}_{2^q}$, but no efficient algorithm was developed to solve the resulting LP problem. Recently, Liu and Draper in [38] extended the binary LP decoding idea in [17] and developed an LP decoding algorithm based on the ADMM technique for nonbinary LDPC codes in $\mathbb{F}_{2^q}$. Specifically, the algorithm is implemented by transforming nonbinary parity-check constraints to an equivalent binary factor graph representation and then relaxing the representation to linear constraints, and finally applying the ADMM algorithm to solve the resulting decoding problem. However, the proposed nonbinary ADMM decoder involves sorting or iteration operations to implement Euclidean projections onto high dimensional parity-check/simplex polytopes, which is time-consuming from a practical viewpoint.

In this paper, we focus on designing new nonbinary LDPC decoders with theoretically-guaranteed convergence, low complexity, and competitive error-correction performance. Specifically, the main contents of this paper are summarized as follows.

- Based on the parity-check equation decomposition method, we decompose a general multi-variables parity-check equation into a set of three-variables parity-check equations. Then, by exploiting the equivalent binary parity-check formulation of every three-variables parity-check equation and Flanagan embedding technique, we transform the nonbinary ML decoding problem to an equivalent linear integer program. Finally, by relaxing binary constraints to box constraints, adding a quadratic penalty term into the objective, and introducing extra linear constraints, a new quadratic programming (QP) decoding model is established for nonbinary LDPC codes in $\mathbb{F}_{2^q}$.
- We develop a proximal-ADMM algorithm to solve the resulting QP decoding model. By exploiting the inherent structures of the QP problem, variables in one ADMM step are blocked and the blocks can be updated in parallel. Moreover, variables in other ADMM steps are updated in parallel.
- The proposed proximal-ADMM decoding algorithm can be proven to converge to a stationary point of the formulated QP decoding problem. In addition, its complexity in each iteration scales linearly with block length and Galois field's size of the nonbinary LDPC codes.
- Besides, we leverage the Constant-Weight embedding technique to develop a different proximal-ADMM decoder, which has a similar convergence property and computational complexity; meanwhile, it satisfies the favorable property of codeword symmetry, i.e., all the transmitted codewords have the same error probability if the noisy channel is symmetrical.

To facilitate reading of this manuscript, notations used in this paper are summarized in Table I. Moreover, we have the following remarks.

- Throughout the paper, all the vectors are column vectors.
- $\mathrm{diag}(\cdot)$ performs different operations depending on its input.
  - If the input is a vector $\mathbf{a}$, $\mathrm{diag}(\mathbf{a})$ denotes a diagonal matrix and $\mathbf{a}$ is its main-diagonal vector.
  - If the input is a square matrix $\mathbf{A}$, $\mathrm{diag}(\mathbf{A}) = \mathbf{a}$, where $\mathbf{a}$ is the matrix's main-diagonal vector.
  - If the input includes vectors or matrices, "$\mathrm{diag}(\cdot,\ldots,\cdot)$" builds a diagonal matrix and the inputs are located in the main-diagonal line of the matrix.
- The operator "$\otimes$" is specifically defined as follows.
  - For matrices $\mathbf{A} \in \mathbb{R}^{m\times n}$ and $\mathbf{B} \in \mathbb{R}^{p\times q}$, $\mathbf{A}\otimes\mathbf{B} = \begin{bmatrix} a_{11}\mathbf{B} & \cdots & a_{1n}\mathbf{B} \\ \vdots & \ddots & \vdots \\ a_{m1}\mathbf{B} & \cdots & a_{mn}\mathbf{B} \end{bmatrix} \in \mathbb{R}^{mp\times nq}$.
  - For column vectors $\mathbf{a} = [a_1,\ldots,a_m]^T$ and $\mathbf{b} = (b_1,\ldots,b_n)^T$, $\mathbf{a}\otimes\mathbf{b} = [a_1\mathbf{b};\ldots;a_m\mathbf{b}] \in \mathbb{R}^{mn}$.
  - For column vector $\mathbf{a} = [a_1,\ldots,a_m]^T$ and matrix $\mathbf{B} \in \mathbb{R}^{p\times q}$, $\mathbf{a}\otimes\mathbf{B} = [a_1\mathbf{B};\ldots;a_m\mathbf{B}] \in \mathbb{R}^{mp\times q}$.

The rest of this paper is organized as follows. In Section II, we briefly introduce the formulation of the ML decoding problem for nonbinary linear block codes. In Section III, we establish a relaxed QP decoding model for nonbinary LDPC codes in $\mathbb{F}_{2^q}$ via decomposition and relaxation techniques of the three-variables parity-check equation. Moreover, an efficient proximal-ADMM algorithm for solving the formulated QP problem is presented in Section IV. Section V shows the convergence and complexity analyses of the proposed proximal-ADMM decoding algorithm.

TABLE I
NOTATIONS AND DESCRIPTIONS

Notations	Descriptions
$\mathbb{F}_{2^q}$	Galois field of characteristic two
$\mathbb{R}$	The set of real numbers
$\mathbf{A}$	Matrix
$\mathbf{a}$	Column vector
a	Scalar
$\{0,1\}^a$	a-length binary column vector
$\{0,1\}^{a\times b}$	a-by-b binary matrix
$[\mathbf{a};\mathbf{b}]$ or $[\mathbf{A};\mathbf{B}]$	Vectors or matrices are concatenated in column-wise
$[\mathbf{a},\mathbf{b}]$ or $[\mathbf{A},\mathbf{B}]$	Vectors or matrices are concatenated in row-wise
$\mathbf{1}_a$ or $\mathbf{1}_{a\times b}$	Length-a all-ones vector or a-by-b all-ones matrix
$\mathbf{0}_a$ or $\mathbf{0}_{a\times b}$	Length-a all-zeros vector or a-by-b all-zeros matrix
$\mathbf{I}_a$	$a\times a$ identity matrix
$(\cdot)^T$	Transpose operator
$\|\cdot\|_2$	ℓ_2-norm
$\preceq$	Generalized inequality
$\otimes$	Kronecker product
$\delta_{\mathbf{A}}$	Spectral norm of matrix $\mathbf{A}$
$\mathrm{diag}(\cdot)$	Vector/matrix diagonalization operator
$\lambda_{\min}(\mathbf{A}^T\mathbf{A})$	Minimum eigenvalue of matrix $\mathbf{A}^T\mathbf{A}$
$\underset{\mathcal{X}}{\Pi}$	Euclidean projection onto set $\mathcal{X}$

Simulation results demonstrate the effectiveness of our proposed decoders for LDPC codes in Section VI. Finally, Section VII concludes this paper.

II. ML Decoding Problem Formulation

This section presents a brief review on the ML decoding problem formulation for nonbinary LDPC codes in $\mathbb{F}_{2^q}$. More details can also be found in [9] [38].

Consider a nonbinary LDPC code defined by an m-by-n check matrix $\mathbf{H}$ in $\mathbb{F}_{2^q}$. Its feasible codeword set $\mathcal{C}$ can be denoted by

$$\mathcal{C}=\left\{\mathbf{c}|\mathbf{h}_j^T\mathbf{c}=0, j\in\mathcal{J}, \mathbf{c}\in\mathbb{F}_{2^q}^n\right\}, \tag{1}$$

where $\mathbf{h}_j^T$, $j\in\mathcal{J}=\{1,2,\cdots,m\}$ denotes the jth row vector of the parity-check matrix $\mathbf{H}$.

Assume that a codeword $\mathbf{c}\in\mathcal{C}$ is transmitted through an additional white Gaussian noise (AWGN) channel and its corresponding output is denoted as $\mathbf{r}\in\mathbb{R}^n$. In the receiver, the aim of ML decoding is to determine which codeword has the largest *a priori* probability $p(\mathbf{r}|\mathbf{c})$ throughout the feasible codeword set $\mathcal{C}$. So, the ML decoding problem can be described as

$$\mathbf{c}^*=\underset{\mathbf{c}\in\mathcal{C}}{\operatorname{argmax}}\ p(\mathbf{r}|\mathbf{c}). \tag{2}$$

Before processing the above ML decoding problem, we introduce two embedding techniques, which can map element c_i in $\mathbb{F}_{2^q}$ to a binary vector in real space.

1) *Flanagan embedding [9]:* the mapped vector $\mathbf{x}_i=[x_{i,1};\cdots;x_{i,2^q-1}]\in\{0,1\}^{2^q-1}$. Specifically, for the nonzero element in $\mathbb{F}_{2^q}$, there are

$$x_{i,\sigma}=\begin{cases}1, & \sigma=c_i,\\ 0, & \sigma\neq c_i,\end{cases} \tag{3}$$

and for the zero element, it is a (2^q-1)-length all-zeros vector.

2) *Constant-Weight embedding [39]:* the mapped vector $\mathbf{x}_i=[x_{i,0};\cdots;x_{i,2^q-1}]\in\{0,1\}^{2^q}$, where

$$x_{i,\sigma}=\begin{cases}1, & \sigma=c_i,\\ 0, & \sigma\neq c_i.\end{cases} \tag{4}$$

Remarks: Both of the above two embedding techniques[1] can be applied to formulating the ML decoding problem (2) to a decoding model in real space. Since most of their derivations are similar, in the following, we only provide the details on how to leverage the former embedding technique. In Appendix E, we provide a brief description and discussion on the latter. Moreover, both of their decoding performances are provided in the simulation section.

When the Flanagan embedding technique is applied, any codeword $\mathbf{c}$ can be mapped to a binary vector $\mathbf{x}=[\mathbf{x}_1;\cdots;\mathbf{x}_n]\in\{0,1\}^{n(2^q-1)}$. Specifically, we call $\mathbf{x}$ as an equivalent binary codeword of the nonbinary codeword $\mathbf{c}$. Let $\mathcal{X}$ denote the set consisting of all equivalent binary codewords. Then, the ML decoding problem (2) can be transformed to

$$\mathbf{x}^*=\underset{\mathbf{x}\in\mathcal{X}}{\operatorname{argmax}}\ p(\mathbf{r}|\mathbf{x}), \tag{5}$$

which can be further derived as

$$\begin{aligned}\underset{\mathbf{x}\in\mathcal{X}}{\operatorname{argmax}}\ p(\mathbf{r}|\mathbf{x}) &= \underset{\mathbf{x}\in\mathcal{X}}{\operatorname{argmax}}\prod_{i=1}^{n}\prod_{\sigma=1}^{2^q-1}p(r_i|x_{i,\sigma})\\ &= \underset{\mathbf{x}\in\mathcal{X}}{\operatorname{argmin}}\sum_{i=1}^{n}\sum_{\sigma=1}^{2^q-1}-\log p(r_i|x_{i,\sigma}).\end{aligned} \tag{6}$$

By adding the constant $\sum_{i=1}^{n}\sum_{\sigma=1}^{2^q-1}\log p(r_i|x_{i,\sigma}=0)$ to (6), we obtain

$$\begin{aligned}\underset{\mathbf{x}\in\mathcal{X}}{\operatorname{argmax}}\ p(\mathbf{r}|\mathbf{x}) &= \underset{\mathbf{x}\in\mathcal{X}}{\operatorname{argmin}}\sum_{i=1}^{n}\sum_{\sigma=1}^{2^q-1}\log\frac{p(r_i|x_{i,\sigma}=0)}{p(r_i|x_{i,\sigma})}\\ &= \underset{\mathbf{x}\in\mathcal{X}}{\operatorname{argmin}}\sum_{i=1}^{n}\sum_{\sigma=1}^{2^q-1}x_{i,\sigma}\log\frac{p(r_i|x_{i,\sigma}=0)}{p(r_i|x_{i,\sigma}=1)}\\ &= \underset{\mathbf{x}\in\mathcal{X}}{\operatorname{argmin}}\ \boldsymbol{\gamma}^T\mathbf{x},\end{aligned} \tag{7}$$

where $\boldsymbol{\gamma}\in\mathbb{R}^{n(2^q-1)}$ is defined by (8), shown at the bottom of the next page. Then, the ML decoding problem (5) can be formulated as the following integer program

$$\min_{\mathbf{x}}\quad \boldsymbol{\gamma}^T\mathbf{x}, \tag{9a}$$

$$\text{s.t.}\quad \mathbf{x}\in\mathcal{X}. \tag{9b}$$

The problem (9) is difficult to solve since constraint (9b) is nonconvex. In the following section, we exploit techniques

[1] *Example:* In $\mathbb{F}_4$, Flanagan embedding: $0\mapsto[0,0,0]$, $1\mapsto[1,0,0]$, $2\mapsto[0,1,0]$, and $3\mapsto[0,0,1]$ and Constant-Weight embedding: $0\mapsto[1,0,0,0]$, $1\mapsto[0,1,0,0]$, $2\mapsto[0,0,1,0]$, and $3\mapsto[0,0,0,1]$.

of decomposition, relaxation, penalty, and tightness to transform (9) to a nonconvex, but tractable quadratic optimization problem.

III. ML Problem Relaxation

In this section, we consider how to relax the ML decoding problem (9) to a tractable QP decoding model for nonbinary LDPC codes in $\mathbb{F}_{2^q}$.

A. Decomposition of the Multi-Variables Parity-Check Equation

Consider the jth parity-check equation in (1). Without loss of generality, we assume it involves $d_j \geq 3$ variables, which are denoted by $c_{\sigma_1}, \cdots, c_{\sigma_{d_j}}$ and the corresponding coefficients are $h_{\sigma_1}, \cdots, h_{\sigma_{d_j}}$. In the following, we show that it can be decomposed equivalently to $d_j - 2$ three-variables parity-check equations by introducing $d_j - 3$ auxiliary variables. The detailed decomposing procedure consists of three steps:

Step 1: for the first two variables c_{σ_1} and c_{σ_2}, we introduce an auxiliary variable g_1 in $\mathbb{F}_{2^q}$ and let them satisfy

$$h_{\sigma_1} c_{\sigma_1} + h_{\sigma_2} c_{\sigma_2} + g_1 = 0. \tag{10}$$

Step 2: for the variables $c_{\sigma_3}, \cdots, c_{\sigma_{d_j-2}}$, we introduce auxiliary variables $g_2, \cdots, g_{d_j-3}$ in $\mathbb{F}_{2^q}$ and let them satisfy

$$g_{t-1} + h_{\sigma_{t+1}} c_{\sigma_{t+1}} + g_t = 0, \quad t = 2, \cdots, d_j - 3. \tag{11}$$

Step 3: let auxiliary variable g_{d_j-3} in $\mathbb{F}_{2^q}$ and the last two variables $c_{\sigma_{d_j-1}}$ and $c_{\sigma_{d_j}}$ satisfy

$$g_{d_j-3} + h_{\sigma_{d_j-1}} c_{\sigma_{d_j-1}} + h_{\sigma_{d_j}} c_{\sigma_{d_j}} = 0. \tag{12}$$

For the above decomposition procedure, we have the following fact:

Fact 1: The set of the three-variables parity-check equations (10)-(12) is equivalent to the jth parity-check equation $\mathbf{h}_j^T \mathbf{c} = 0$ in (1) in the sense that their solutions are one-to-one correspondent.

Proof: See Appendix A. ∎

Applying (10)-(12) to all of the parity-check equations in (1), the total numbers of the three-variables parity-check equations in $\mathbb{F}_{2^q}$ and the introduced auxiliary variables are

$$\Gamma_c = \sum_{j=1}^{m} (d_j - 2), \quad \Gamma_a = \sum_{j=1}^{m} (d_j - 3), \tag{13}$$

respectively.

In the following, we show that any three-variables parity-check equation in $\mathbb{F}_{2^q}$ has an equivalent expression in real space.

B. Equivalent Expression of the Three-Variables Parity-Check Equation in Real Space

Consider the following three-variables parity-check equation in $\mathbb{F}_{2^q}$

$$\sum_{k=1}^{3} h_k c_k = 0, \tag{14}$$

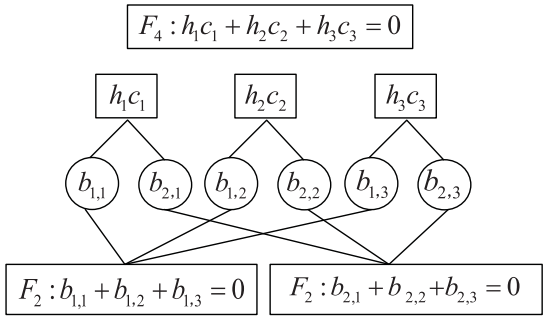


Fig. 1. The factor graph representation for equivalence between the three-variables parity-check equation in $\mathbb{F}_4$ and the two three-variables parity-check equations in $\mathbb{F}_2$, where $h_1, h_2, h_3 \in \mathbb{F}_4 \setminus 0$, $c_1, c_2, c_3 \in \mathbb{F}_4$ and $b_{1,1}, b_{2,1}, b_{1,2}, b_{2,2}, b_{1,3}, b_{2,3} \in \mathbb{F}_2$.

where $h_k, c_k \in \mathbb{F}_{2^q}$ but h_k is nonzero. Since $h_k c_k \in \mathbb{F}_{2^q}$ can be expressed exactly as[2]

$$\mathcal{F}(h_k c_k) = \sum_{i=1}^{q} b_{i,k} \zeta^{i-1}, \tag{15}$$

where $b_{i,k} \in \mathbb{F}_2$, ζ is the primitive element in $\mathbb{F}_{2^q}$, and function $\mathcal{F}(\cdot)$ denotes $h_k c_k$'s polynomial representation in $\mathbb{F}_{2^q}$. Then, (14) is equivalent to the following q three-variables parity-check equations in $\mathbb{F}_2$

$$\sum_{k=1}^{3} b_{i,k} = 0, \ i = 1, 2, \cdots, q.^3 \tag{16}$$

Figure 1 shows an example of equivalence between a three-variables parity-check equation in $\mathbb{F}_4$ and two three-variables parity-check equations in $\mathbb{F}_2$.

Next, we consider how to obtain equivalent expression of the parity-check equations (16) in real space. First, notice any nonzero element $h_k \in \mathbb{F}_{2^q}$ can be mapped to an elementary matrix $\mathbf{D}(2^q, h_k) \in \{0,1\}^{(2^q-1)\times(2^q-1)}$ [38], whose entries are defined by[4]

$$D(2^q, h_k)_{ij} = \begin{cases} 1, & \text{if } i = j \cdot h_k, \\ 0, & \text{otherwise}, \end{cases} \tag{17}$$

where the multiplication in $j \cdot h_k$ is in $\mathbb{F}_{2^q}$. Then, we have the following lemma.

[2]Different representations of nonzero elements in $\mathbb{F}_4$:

	polynomial	bits	integer
ζ^0	1	01	1
ζ^1	ζ	10	2
ζ^2	$\zeta + 1$	11	3

[3]*Example*: Consider $1 \cdot 2 + 2 \cdot 2 + 3 \cdot 2 = 0$ in $\mathbb{F}_4$. Since $1 \cdot 2 = 2$, $2 \cdot 2 = 3$, $3 \cdot 2 = 1$, and $\mathcal{F}(2) = \zeta$, $\mathcal{F}(3) = 1 + \zeta$, and $\mathcal{F}(1) = 1$ respectively, we see that the corresponding coefficient vectors of the polynomials are $[b_{1,1}, b_{2,1}] = [0, 1]$, $[b_{1,2}, b_{2,2}] = [1, 1]$, and $[b_{1,3}, b_{2,3}] = [1, 0]$ respectively. Then, one can see that $1 \cdot 2 + 2 \cdot 2 + 3 \cdot 2 = 0$ in $\mathbb{F}_4$ is equivalent to two parity-check equations in $\mathbb{F}_2$: $0 + 1 + 1 = 0$ and $1 + 1 + 0 = 0$.

[4]*Example:* In $\mathbb{F}_4$, $\mathbf{D}(4,1) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$, $\mathbf{D}(4,2) = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$, and $\mathbf{D}(4,3) = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$.

$$\boldsymbol{\gamma} = \left[\log \frac{p(r_1|x_{1,1}=0)}{p(r_1|x_{1,1}=1)}, \cdots, \log \frac{p(r_1|x_{1,2^q-1}=0)}{p(r_1|x_{1,2^q-1}=1)}, \cdots, \log \frac{p(r_n|x_{n,1}=0)}{p(r_n|x_{n,1}=1)}, \cdots, \log \frac{p(r_n|x_{n,2^q-1}=0)}{p(r_n|x_{n,2^q-1}=1)} \right]. \tag{8}$$

Lemma 1: Let $\mathbf{x}_k$ be c_k's equivalent binary codeword according to mapping rule (4). Then, $\mathbf{D}(2^q, h_k)\mathbf{x}_k$ is an equivalent binary codeword to $h_k c_k$ mapped according to the same rule.

Proof: See Appendix B. ∎

Let binary vector $\tilde{\mathbf{b}}(\tilde{\alpha}) = [\tilde{b}_1; \cdots; \tilde{b}_q]$ be bits expression for any nonzero element $\tilde{\alpha} \in \mathbb{F}_{2^q}$ and formulate the following q-by-(2^q-1) binary matrix[5]

$$\mathbf{B} = \left[\tilde{\mathbf{b}}(1), \tilde{\mathbf{b}}(2), \ldots, \tilde{\mathbf{b}}(2^q-1)\right] = \begin{bmatrix} 1 & 0 & \cdots & 0 & 1 \\ 0 & 1 & \cdots & 1 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & 1 \end{bmatrix}. \quad (18)$$

Let $\hat{\mathbf{b}}_i^T$ denote the ith row vector of matrix $\mathbf{B}$. Then, we can obtain[6]

$$b_{i,k} = \hat{\mathbf{b}}_i^T \mathbf{D}(2^q, h_k)\mathbf{x}_k, \ \forall i = 1, 2, \ldots, q; \ k = 1, 2, 3. \quad (19)$$

Note that the multiplications/additions in (19) operate in real space, but the result can only be 1 or 0 since matrix $\mathbf{D}(2^q, h_k)$ is elementary and $\mathbf{x}_k$ only includes at most one nonzero element 1. It means $b_{i,k}$ lies in $\mathbb{F}_2$. Therefore, three-variables parity-check equations (16) in $\mathbb{F}_2$ can be rewritten as

$$\hat{\mathbf{b}}_i^T \mathbf{D}(2^q, h_1)\mathbf{x}_1 + \hat{\mathbf{b}}_i^T \mathbf{D}(2^q, h_2)\mathbf{x}_2 + \hat{\mathbf{b}}_i^T \mathbf{D}(2^q, h_3)\mathbf{x}_3 = 0, \quad (20)$$

where the sum is in $\mathbb{F}_2$ and $i = 1, 2, \ldots, q$. Moreover, three-variables parity-check equations (16) can be equivalent to the following inequality system defined in real space

$$\begin{aligned} &f_{i,1} \le f_{i,2} + f_{i,3}, \quad f_{i,2} \le f_{i,1} + f_{i,3}, \\ &f_{i,3} \le f_{i,1} + f_{i,2}, \quad f_{i,1} + f_{i,2} + f_{i,3} \le 2, \\ &f_{i,1}, f_{i,2}, f_{i,3} \in \{0, 1\}, \ i = 1, 2, \cdots, q, \end{aligned} \quad (21)$$

in the sense that solutions $b_{i,k}$ and $f_{i,k}$ are one-to-one correspondent.[7] Let

$$\mathbf{t} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 2 \end{bmatrix}, \quad \mathbf{P} = \begin{bmatrix} 1 & -1 & -1 \\ -1 & 1 & -1 \\ -1 & -1 & 1 \\ 1 & 1 & 1 \end{bmatrix}. \quad (22)$$

Then, (21) can be rewritten as

$$\mathbf{P}\mathbf{f}_i \preceq \mathbf{t}, \ \mathbf{f}_i \in \{0, 1\}^3, \ i = 1, 2, \ldots, q, \quad (23)$$

where $\mathbf{f}_i = [f_{i,1}; f_{i,2}; f_{i,3}]$. Notice that $f_{i,k}$ can take the place of the kth term $b_{i,k}$ in (20). Therefore, define

$$\mathbf{T}_i = \mathrm{diag}(\hat{\mathbf{b}}_i^T, \hat{\mathbf{b}}_i^T, \hat{\mathbf{b}}_i^T) \in \{0, 1\}^{3 \times 3(2^q-1)}, \quad (24a)$$

$$\mathbf{D} = \mathrm{diag}(\mathbf{D}(2^q, h_1), \mathbf{D}(2^q, h_2), \mathbf{D}(2^q, h_3)) \in \{0, 1\}^{3(2^q-1) \times 3(2^q-1)}, \quad (24b)$$

$$\mathbf{x} = [\mathbf{x}_1; \mathbf{x}_2; \mathbf{x}_3] \in \{0, 1\}^{3(2^q-1)}. \quad (24c)$$

We can obtain

$$\mathbf{f}_i = \mathbf{T}_i \mathbf{D}\mathbf{x}. \quad (25)$$

Then, plugging (25) into (23), we have

$$\mathbf{P}\mathbf{T}_i\mathbf{D}\mathbf{x} \preceq \mathbf{t}, \ \forall i \in \{1, 2, \ldots, q\}. \quad (26)$$

Furthermore, let

$$\mathbf{W} = [\mathbf{P}\mathbf{T}_1\mathbf{D}; \ldots; \mathbf{P}\mathbf{T}_q\mathbf{D}] \in \mathbb{R}^{4q \times 3(2^q-1)}, \quad (27a)$$

$$\mathbf{w} = \mathbf{1}_q \otimes \mathbf{t} \in \mathbb{R}^{4q}. \quad (27b)$$

Then, the three-variables parity-check equation (14) can be equivalent to

$$\mathbf{W}\mathbf{x} \preceq \mathbf{w}, \ \mathbf{x} \in \{0, 1\}^{3(2^q-1)} \quad (28)$$

in the sense that their solutions $\mathbf{c}$ and $\mathbf{x}$ are one-to-one correspondent under the mapping rule (4).

C. Equivalent ML Decoding Problem

In this subsection, we consider to establish a linear integer program equivalent to the ML decoding problem (9) for nonbinary LDPC codes in $\mathbb{F}_{2^q}$.

First, we define

$$\mathbf{v} = [\mathbf{x}; \mathbf{s}] \in \{0, 1\}^{(2^q-1)(n+\Gamma_a)}, \quad (29)$$

where $\mathbf{s} = [\mathbf{s}_1; \cdots; \mathbf{s}_i; \cdots; \mathbf{s}_{\Gamma_a}] \in \{0, 1\}^{(2^q-1)\Gamma_a}$ and $\mathbf{s}_i \in \{0, 1\}^{2^q-1}$ correspond to the ith auxiliary variable introduced in the decomposition procedure (10)-(12). Define a variable-selecting matrix $\mathbf{Q}_\tau \in \{0, 1\}^{3 \times (n+\Gamma_a)}$ corresponding to the τth decomposed three-variables parity-check equation in $\mathbb{F}_{2^q}$ for parity-check equations in (1), where $\tau = 1, \cdots, \Gamma_c$. Its every row vector includes only one "1", whose index corresponds to the variable in (1).[8] Therefore, $(\mathbf{Q}_\tau \otimes \mathbf{I}_{2^q-1})\mathbf{v}$ are the variables involved in the τth three-variables parity-check equation in $\mathbb{F}_2$. Moreover, we define

$$\begin{aligned} \mathbf{F} = [&\mathbf{W}_1(\mathbf{Q}_1 \otimes \mathbf{I}_{2^q-1}); \cdots; \mathbf{W}_\tau(\mathbf{Q}_\tau \otimes \mathbf{I}_{2^q-1}); \cdots; \\ &\mathbf{W}_{\Gamma_c}(\mathbf{Q}_{\Gamma_c} \otimes \mathbf{I}_{2^q-1})] \in \mathbb{R}^{4q\Gamma_c \times (n+\Gamma_a)(2^q-1)}, \end{aligned} \quad (30a)$$

$$\mathbf{u} = \mathbf{1}_{\Gamma_c} \otimes \mathbf{w} \in \mathbb{R}^{4q\Gamma_c}. \quad (30b)$$

Then, by (28), we have

$$\mathbf{F}\mathbf{v} \preceq \mathbf{u}. \quad (31)$$

Therefore, we can transform the ML decoding problem (9) to the following linear integer program

$$\min_{\mathbf{v}} \quad \boldsymbol{\lambda}^T \mathbf{v}, \quad (32a)$$

$$\text{s.t.} \quad \mathbf{F}\mathbf{v} \preceq \mathbf{u}, \quad (32b)$$

$$\mathbf{v} \in \{0, 1\}^{(2^q-1)(n+\Gamma_a)}, \quad (32c)$$

where $\boldsymbol{\lambda} = [\boldsymbol{\gamma}; \mathbf{0}_{\Gamma_a(2^q-1)}] \in \mathbb{R}^{(2^q-1)(n+\Gamma_a)}$.

Due to the binary constraints (32c), problem (32) is difficult to solve. In the following, we relax it to a continuous, nonconvex, but tractable model.

[5] *Example:* In $\mathbb{F}_4$, $\mathbf{B} = \left[\tilde{\mathbf{b}}(1), \tilde{\mathbf{b}}(2), \tilde{\mathbf{b}}(3)\right] = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$.

[6] *Example:* To be clear, consider multiplication $3 \cdot 2 = 1$ in $\mathbb{F}_4$: according to Lemma 1, equivalent binary codeword of $3 \cdot 2$ can be determined by $\begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$. Moreover, $\mathbf{B} \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$.

[7] The one-to-one correspondence can be seen clearly from the following example: assume that $[b_{1,1}, b_{2,1}] = [1, 0]$, $[b_{1,2}, b_{2,2}] = [0, 1]$, and $[b_{1,3}, b_{2,3}] = [1, 1]$. It is easy to check that they satisfy (16). Moreover, it is obvious that reals $[f_{1,1}, f_{2,1}] = [1, 0]$, $[f_{1,2}, f_{2,2}] = [0, 1]$, and $[f_{1,3}, f_{2,3}] = [1, 1]$ satisfy the inequality system (21). This example shows that if $b_{i,k} = 1$, then $f_{i,k} = 1$, or if $b_{i,k} = 0$, then $f_{i,k} = 0$. Notice $b_{i,k} \in \mathbb{F}_2$ and $f_{i,k}$ are in real space.

[8] *Example*: Consider the parity-check equation in $\mathbb{F}_4$: $c_1 + c_2 + 2c_3 + 3c_4 = 0$. According to (10)-(12), it can be decomposed to two three-variables parity-check equations: $g_1 + c_1 + c_2 = 0$, $g_1 + 2c_3 + 3c_4 = 0$. Then, the corresponding variable-selecting matrices $\mathbf{Q}_1 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$ and $\mathbf{Q}_2 = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$.

D. Proposed Decoding Model

In this subsection, we exploit a simple relaxation method for the linear integer program (9) and then introduce three techniques to alleviate the *relaxation* effect on the optimal solution, which leads to the proposed decoding model for nonbinary LDPC codes in $\mathbb{F}_{2^q}$.

The typical way for the binary constraint (32c) is to relax it to a box constraint, i.e., $\mathbf{v} \in [0,1]^{(2^q-1)(n+\Gamma_a)}$, which can simplify the nonconvex problem (32) to a convex one. However, the resulting optimization problem's optimal solution could be fractional especially when the decoder works in low SNR regions. In the following, we deploy three techniques to handle this problem.

The first one is to add a quadratic penalty term into the objective, i.e.,

$$\boldsymbol{\lambda}^T\mathbf{v} - \frac{\alpha}{2}\|\mathbf{v}-0.5\|_2^2, \tag{33}$$

where $\alpha > 0$ is a preset constant. Empirically, the quadratic penalty makes the optimal integer solutions favorable and improves error-correction performance of LDPC decoders significantly [10] [30] [38].

The second one is to introduce extra linear constraints to tighten the relaxation. Specifically, let $\{\mathcal{K}_\ell | \ell = 1, \cdots, 2^q-1\}$ denote all the subsets of the set $\{1, \cdots, q\}$. Then, (20) can be equivalent to (34), shown at the bottom of the page, where the sum $\sum_{i\in\mathcal{K}_\ell}$ is in $\mathbb{F}_2$. Define

$$\hat{\mathbf{T}}_\ell = \mathrm{diag}\Big(\sum_{i\in\mathcal{K}_\ell}\hat{\mathbf{b}}_i^T, \sum_{i\in\mathcal{K}_\ell}\hat{\mathbf{b}}_i^T, \sum_{i\in\mathcal{K}_\ell}\hat{\mathbf{b}}_i^T\Big), \tag{35a}$$

$$\hat{\mathbf{W}} = [\mathbf{P}\hat{\mathbf{T}}_1\mathbf{D}; \ldots; \mathbf{P}\hat{\mathbf{T}}_{2^q-1}\mathbf{D}] \in \mathbb{R}^{4(2^q-1)\times 3(2^q-1)}, \tag{35b}$$

$$\hat{\mathbf{w}} = \mathbf{1}_{2^q-1} \otimes \mathbf{t} \in \mathbb{R}^{4(2^q-1)}, \tag{35c}$$

Then, according to (21), we can obtain (34)'s equivalent inequalities system in real space as follows:

$$\hat{\mathbf{W}}\mathbf{x} \preceq \hat{\mathbf{w}}, \quad \mathbf{x} \in \{0,1\}^{3(2^q-1)}. \tag{36}$$

Since $\hat{\mathbf{w}}$ is a $4(2^q-1)$-length vector and $\hat{\mathbf{W}}$ is a $4(2^q-1)$-by-$3(2^q-1)$ matrix, (36) consists of $4(2^q-1)$ inequalities. Besides $4q$ inequalities same as in (28), other inequalities in (36) can be seen as redundant ones since they are combined by inequalities in (28). However, when the binary constraint $\mathbf{x} \in \{0,1\}^{3(2^q-1)}$ is relaxed to the box constraint $\mathbf{x} \in [0,1]^{3(2^q-1)}$, these redundant inequalities can play a role in tightening the relaxation. By defining

$$\hat{\mathbf{F}} = [\hat{\mathbf{W}}_1(\mathbf{Q}_1 \otimes \mathbf{I}_{2^q-1}); \cdots; \hat{\mathbf{W}}_\tau(\mathbf{Q}_\tau \otimes \mathbf{I}_{2^q-1}); \cdots; \hat{\mathbf{W}}_{\Gamma_c}(\mathbf{Q}_{\Gamma_c} \otimes \mathbf{I}_{2^q-1})] \in \mathbb{R}^{4(2^q-1)\Gamma_c \times (n+\Gamma_a)(2^q-1)}, \tag{37a}$$

$$\hat{\mathbf{u}} = \mathbf{1}_{\Gamma_c} \otimes \hat{\mathbf{w}} \in \mathbb{R}^{4(2^q-1)\Gamma_c}. \tag{37b}$$

we can transform (31) equivalently to

$$\hat{\mathbf{F}}\mathbf{v} \preceq \hat{\mathbf{u}}. \tag{38}$$

The third one is used to exploit the structure so that the equivalent binary codeword of the element in $\mathbb{F}_{2^q}$ has at most one 1 (see (4)). To do so, we define a binary matrix

$$\mathbf{S} = \mathrm{diag}(\underbrace{\mathbf{1}_{2^q-1}^T, \cdots, \mathbf{1}_{2^q-1}^T}_{n+\Gamma_a}) \in \{0,1\}^{(n+\Gamma_a)\times(2^q-1)(n+\Gamma_a)}, \tag{39}$$

which leads to

$$\mathbf{S}\mathbf{v} \preceq \mathbf{1}_{n+\Gamma_a}. \tag{40}$$

Then, by combining (33), (38), and (40), we relax the ML decoding problem (9) to

$$\min_{\mathbf{v}} \quad \boldsymbol{\lambda}^T\mathbf{v} - \frac{\alpha}{2}\|\mathbf{v}-0.5\|_2^2, \tag{41a}$$

$$\text{s.t.} \quad \hat{\mathbf{F}}\mathbf{v} \preceq \hat{\mathbf{u}}, \tag{41b}$$

$$\mathbf{S}\mathbf{v} \preceq \mathbf{1}_{n+\Gamma_a}, \tag{41c}$$

$$\mathbf{v} \in [0,1]^{(2^q-1)(n+\Gamma_a)}. \tag{41d}$$

In the next section, we present an efficient solving algorithm via the proximal-ADMM technique for the decoding problem (43), where variables in every ADMM iteration are solved analytically and in parallel. Moreover, we show the proposed decoder is theoretically-guaranteed convergent to a stationary point of the problem (41) and its computational complexity in each iteration scales linearly with code length and size of the Galois field.

IV. Proximal-ADMM Solving Algorithm

In this section, we develop a proximal-ADMM algorithm to solve the decoding problem (41). Moreover, by exploiting its inherent structures, each subproblem in the proximal-ADMM iteration can be solved efficiently.

A. Proximal-ADMM Algorithm Framework

Define

$$\mathbf{A} = [\hat{\mathbf{W}}_1(\mathbf{Q}_1 \otimes \mathbf{I}_{2^q-1}); \cdots; \hat{\mathbf{W}}_\tau(\mathbf{Q}_\tau \otimes \mathbf{I}_{2^q-1}); \cdots; \hat{\mathbf{W}}_{\Gamma_c}(\mathbf{Q}_{\Gamma_c} \otimes \mathbf{I}_{2^q-1}); \mathbf{S}] \in \mathbb{R}^{M\times N}, \tag{42a}$$

$$\boldsymbol{\varrho} = [\hat{\mathbf{u}}; \mathbf{1}_{n+\Gamma_a}] \in \mathbb{R}^M, \tag{42b}$$

where $M = 4(2^q-1)\Gamma_c + n + \Gamma_a$ and $N = (n+\Gamma_a)(2^q-1)$. Then, we can transform (41) to

$$\min_{\mathbf{v}} \quad \boldsymbol{\lambda}^T\mathbf{v} - \frac{\alpha}{2}\|\mathbf{v}-0.5\|_2^2, \tag{43a}$$

$$\text{s.t.} \quad \mathbf{A}\mathbf{v} \preceq \boldsymbol{\varrho}, \tag{43b}$$

$$\mathbf{v} \in [0,1]^{(2^q-1)(n+\Gamma_a)}. \tag{43c}$$

Moreover, by introducing two auxiliary variables, $\mathbf{e}_1$ and $\mathbf{e}_2$, the decoding problem (43) is equivalent to

$$\min_{\mathbf{v},\mathbf{e}_1,\mathbf{e}_2} \quad \boldsymbol{\lambda}^T\mathbf{v} - \frac{\alpha}{2}\|\mathbf{v}-0.5\|_2^2, \tag{44a}$$

$$\text{s. t.} \quad \mathbf{A}\mathbf{v} + \mathbf{e}_1 = \boldsymbol{\varrho}, \tag{44b}$$

$$\mathbf{v} = \mathbf{e}_2, \tag{44c}$$

$$\mathbf{e}_1 \succeq \mathbf{0}_M, \ \mathbf{0}_N \preceq \mathbf{e}_2 \preceq \mathbf{1}_N. \tag{44d}$$

$$\Big(\sum_{i\in\mathcal{K}_\ell}\hat{\mathbf{b}}_i^T\Big)\mathbf{D}(2^q,h_1)\mathbf{x}_1 + \Big(\sum_{i\in\mathcal{K}_\ell}\hat{\mathbf{b}}_i^T\Big)\mathbf{D}(2^q,h_2)\mathbf{x}_2 + \Big(\sum_{i\in\mathcal{K}_\ell}\hat{\mathbf{b}}_i^T\Big)\mathbf{D}(2^q,h_3)\mathbf{x}_3 = 0, \quad \ell = 1,\ldots,2^q-1. \tag{34}$$

Its augmented Lagrangian function can be written as

$$\mathcal{L}_\mu(\mathbf{v},\mathbf{e}_1,\mathbf{e}_2,\mathbf{y}_1,\mathbf{y}_2)=\boldsymbol{\lambda}^T\mathbf{v}-\frac{\alpha}{2}\|\mathbf{v}-0.5\|_2^2+\mathbf{y}_1^T(\mathbf{A}\mathbf{v}+\mathbf{e}_1-\boldsymbol{\varrho})+\mathbf{y}_2^T(\mathbf{v}-\mathbf{e}_2)+\frac{\mu}{2}\|\mathbf{A}\mathbf{v}+\mathbf{e}_1-\boldsymbol{\varrho}\|_2^2+\frac{\mu}{2}\|\mathbf{v}-\mathbf{e}_2\|_2^2, \tag{45}$$

where $\mathbf{y}_1 \in \mathbb{R}^M$ and $\mathbf{y}_2 \in \mathbb{R}^N$ are Lagrangian multipliers corresponding to equality constraints in (44b) and (44c) respectively and $\mu > 0$ is a preset penalty parameter. Based on the augmented Lagrangian (45), the proximal-ADMM solving algorithm for model (44) can be described as follows

$$\mathbf{v}^{k+1}=\arg\min_{\mathbf{v}} \mathcal{L}_\mu(\mathbf{v},\mathbf{e}_1^k,\mathbf{e}_2^k,\mathbf{y}_1^k,\mathbf{y}_2^k)+\frac{\rho}{2}\|\mathbf{v}-\mathbf{p}^k\|_2^2, \tag{46a}$$

$$\mathbf{e}_1^{k+1}=\arg\min_{\mathbf{e}_1\succeq\mathbf{0}_M} \mathcal{L}_\mu(\mathbf{v}^{k+1},\mathbf{e}_1,\mathbf{e}_2^k,\mathbf{y}_1^k,\mathbf{y}_2^k)+\frac{\rho}{2}\|\mathbf{e}_1-\mathbf{z}_1^k\|_2^2, \tag{46b}$$

$$\mathbf{e}_2^{k+1}=\arg\min_{\mathbf{0}_N\preceq\mathbf{e}_2\preceq\mathbf{1}_N} \mathcal{L}_\mu(\mathbf{v}^{k+1},\mathbf{e}_1^k,\mathbf{e}_2,\mathbf{y}_1^k,\mathbf{y}_2^k)+\frac{\rho}{2}\|\mathbf{e}_2-\mathbf{z}_2^k\|_2^2, \tag{46c}$$

$$\mathbf{p}^{k+1}=\mathbf{p}^k+\beta(\mathbf{v}^{k+1}-\mathbf{p}^k), \tag{46d}$$

$$\mathbf{z}_1^{k+1}=\mathbf{z}_1^k+\beta(\mathbf{e}_1^{k+1}-\mathbf{z}_1^k), \tag{46e}$$

$$\mathbf{z}_2^{k+1}=\mathbf{z}_2^k+\beta(\mathbf{e}_2^{k+1}-\mathbf{z}_2^k), \tag{46f}$$

$$\mathbf{y}_1^{k+1}=\mathbf{y}_1^k+\mu(\mathbf{A}\mathbf{v}^{k+1}+\mathbf{e}_1^{k+1}-\boldsymbol{\varrho}), \tag{46g}$$

$$\mathbf{y}_2^{k+1}=\mathbf{y}_2^k+\mu(\mathbf{v}^{k+1}-\mathbf{e}_2^{k+1}), \tag{46h}$$

where k is the iteration number, $\|\mathbf{v}-\mathbf{p}^k\|_2^2$, $\|\mathbf{e}_1-\mathbf{z}_1^k\|_2^2$, and $\|\mathbf{e}_2-\mathbf{z}_2^k\|_2^2$ are the so-called proximal terms, ρ is the corresponding penalty parameter, and β belongs to $(0,1]$. In the above iterations, variables $\mathbf{p}$, $\mathbf{z}_1$, and $\mathbf{z}_2$ are generated by an exponential averaging (or smoothing) scheme. Since extra quadratic proximal terms, centered at $\mathbf{p}$, $\mathbf{z}_1$, and $\mathbf{z}_2$, are inserted into the augmented Lagrangian function with respect to variables $\mathbf{x}$, $\mathbf{e}_1$, and $\mathbf{e}_2$ intuitively, $\mathbf{x}^{k+1}$, $\mathbf{e}_1^{k+1}$ and $\mathbf{e}_2^{k+1}$ may not deviate too much from the stabilized iterates $\mathbf{p}^k$, $\mathbf{z}_1^k$, and $\mathbf{z}_2^k$ respectively(see more details in [44]).

In the following, we show that subproblems (46a)-(46c) can be solved efficiently by exploiting the inherent structures of the model.

B. Solving Subproblem (46a)

Obviously, choosing the values of parameters ρ, μ, and α properly, problem (46a) can be reduced to a strongly quadratic convex one with respect to $\mathbf{v}$. In this case, its solving procedure can be described as follows: by setting the gradient of the function $\mathcal{L}_\mu(\mathbf{v},\mathbf{u}_1^k,\mathbf{u}_2^k,\mathbf{y}_1^k,\mathbf{y}_2^k)+\frac{\rho}{2}\|\mathbf{v}-\mathbf{p}^k\|_2^2$ to be zero and solving the corresponding linear equation, we update $\mathbf{v}$ as

$$\mathbf{v}^{k+1}=\left(\mathbf{A}^T\mathbf{A}+\epsilon\mathbf{I}_N\right)^{-1}\boldsymbol{\varphi}^k, \tag{47}$$

where

$$\epsilon=1+\frac{\rho}{\mu}-\frac{\alpha}{\mu}, \tag{48a}$$

$$\boldsymbol{\varphi}^k=\mathbf{A}^T\left(\boldsymbol{\varrho}-\mathbf{e}_1^k-\frac{\mathbf{y}_1^k}{\mu}\right)+\left(\mathbf{e}_2^k-\frac{\mathbf{y}_2^k}{\mu}\right)+\frac{\rho}{\mu}\mathbf{p}^k-\frac{\boldsymbol{\lambda}+0.5\alpha}{\mu}. \tag{48b}$$

Note that $\left(\mathbf{A}^T\mathbf{A}+\epsilon\mathbf{I}_N\right)^{-1}$ is fixed for a given nonbinary LDPC code. Thus, it only needs to be calculated only once throughout the proximal-ADMM iterations. Therefore, the main computational cost lies in $\left(\mathbf{A}^T\mathbf{A}+\epsilon\mathbf{I}_N\right)^{-1}\boldsymbol{\varphi}^k$, which requires $\mathcal{O}\big((2^q-1)^2(n+\Gamma_a)^2\big)$ multiplications. It is prohibitive for large-scale problems in practice. In the following, we show a much more efficient way to perform the computational procedure:

Lemma 2: Matrix $\left(\mathbf{A}^T\mathbf{A}+\epsilon\mathbf{I}_N\right)^{-1}$ is block diagonal. Specifically, it can be denoted by

$$\left(\mathbf{A}^T\mathbf{A}+\epsilon\mathbf{I}_N\right)^{-1}=\mathrm{diag}\Big(\left(\mathbf{A}_1^T\mathbf{A}_1+\epsilon\mathbf{I}_{2^q-1}\right)^{-1},\ldots,\left(\mathbf{A}_{n+\Gamma_a}^T\mathbf{A}_{n+\Gamma_a}+\epsilon\mathbf{I}_{2^q-1}\right)^{-1}\Big), \tag{49}$$

where sub-matrix $\mathbf{A}_i$, $\forall i \in \{1,\cdots,n+\Gamma_a\}$, is formed by column vectors indexed from $\big((2^q-1)(i-1)+1\big)$ to $(2^q-1)i$ in matrix $\mathbf{A}$. Moreover,

$$\left(\mathbf{A}_i^T\mathbf{A}_i+\epsilon\mathbf{I}_{2^q-1}\right)^{-1}=\begin{bmatrix}\theta_i & \omega_i & \cdots & \omega_i\\ \omega_i & \theta_i & \cdots & \omega_i\\ \vdots & \vdots & \vdots & \vdots\\ \omega_i & \omega_i & \cdots & \theta_i\end{bmatrix}, \tag{50}$$

where

$$\omega_i=\frac{-2^qd_i-1}{(2^qd_i+\epsilon)\big((2^{q+1}d_i+\epsilon+1)+(2^qd_i+1)(2^q-2)\big)},$$
$$\theta_i=\omega_i+\frac{1}{2^qd_i+\epsilon}. \tag{51}$$

Here, d_i denotes the degree of the ith information symbol, i.e., the number of check equations involved.

Proof: See Appendix C. ∎

The equation (49) indicates that $\mathbf{v}^{k+1}$ in (47) can be obtained through the following parallel implementations

$$\mathbf{v}_i^{k+1}=\left(\mathbf{A}_i^T\mathbf{A}_i+\epsilon\mathbf{I}_{2^q-1}\right)^{-1}\boldsymbol{\varphi}_i^k, \forall i\in\{1,2,\cdots,n+\Gamma_a\}, \tag{52}$$

where $\boldsymbol{\varphi}_i^k$ is the corresponding (2^q-1)-length sub-vector in $\boldsymbol{\varphi}^k$. Moreover, based on (50), (52) can be further derived as follows:

$$\begin{aligned}\mathbf{v}_i^{k+1}&=\begin{bmatrix}\theta_i-\omega_i & 0 & \cdots & 0\\ 0 & \theta_i-\omega_i & \cdots & 0\\ \vdots & \vdots & \vdots & \vdots\\ 0 & 0 & \cdots & \theta_i-\omega_i\end{bmatrix}\boldsymbol{\varphi}_i^k\\ &\quad+\begin{bmatrix}\omega_i & \omega_i & \cdots & \omega_i\\ \omega_i & \omega_i & \cdots & \omega_i\\ \vdots & \vdots & \vdots & \vdots\\ \omega_i & \omega_i & \cdots & \omega_i\end{bmatrix}\boldsymbol{\varphi}_i^k\\ &=(\theta_i-\omega_i)\boldsymbol{\varphi}_i^k+\big(\omega_i\sum_{\iota=1}^{2^q-1}\varphi_{i,\iota}^k\big)\mathbf{1}_{2^q-1},\end{aligned} \tag{53}$$

where $\varphi_{i,\iota}^k$ denotes the ιth entry in $\boldsymbol{\varphi}_i^k$.

C. Solving Subproblems (46b) *and* (46c)

Solving (46b) is equivalent to solving the following M subproblems in parallel

$$\min_{e_{1,j}} \ y_{1,j}^k e_{1,j}+\frac{\mu}{2}\big(\mathbf{a}_j^T\mathbf{v}^{k+1}+e_{1,j}-\varrho_j\big)^2+\frac{\rho}{2}\left(e_{1,j}-z_{1,j}^k\right)^2$$
$$\text{s.t.}\quad e_{1,j}\geq 0,\ \forall j\in\{1,\cdots,M\}, \tag{54}$$

Algorithm 1 Proximal-ADMM Decoding Algorithm

1: Initializations: decompose each check equation into three-variables parity-check equations based on the parity-check matrix $\mathbf{H}$. Construct matrix $\mathbf{T}_i$ and $\mathbf{D}$ via (24a) and (24b) respectively. Construct matrix $\mathbf{B}$ via (18) and the variable-selecting matrices $\mathbf{Q}_\tau$, $\tau = 1, \ldots, \Gamma_c$. Construct matrices $\hat{\mathbf{W}}_j$, $j = 1, \ldots, \Gamma_c$, via (35). Construct $\mathbf{A}$ and $\boldsymbol{\varrho}$ via (42). Let $\mu > 0$, $\rho > \alpha$ and $0 < \beta \leq 1$. For all $i \in \{1, \cdots, n+\Gamma_a\}$, compute θ_i and ω_i via (51). Initialize variables $\{\mathbf{v}, \mathbf{e}_1, \mathbf{e}_2, \mathbf{p}, \mathbf{z}_1, \mathbf{z}_2, \mathbf{y}_1, \mathbf{y}_2\}$ as all-zeros vectors.
2: **Repeat**
3: Compute $\boldsymbol{\varphi}^k$ via (48). Then, update $\mathbf{v}_i^{k+1}$ via (53) in parallel, $\forall i \in \{1, \ldots, n+\Gamma_a\}$.
4: Update $e_{1,j}^{k+1}$, $\forall j \in \{1, \ldots, M\}$, via (55) in parallel.
5: Update $e_{2,\ell}^{k+1}$, $\forall \ell \in \{1, \ldots, N\}$, via (57) in parallel.
6: Update $\mathbf{p}^{k+1}$, $\mathbf{z}_1^{k+1}$, $\mathbf{z}_2^{k+1}$, $\mathbf{y}_1^{k+1}$ and $\mathbf{y}_2^{k+1}$ in parallel via (46e) – (46h) respectively.
7: $k \leftarrow k+1$.
8: **Until** some preset conditions are satisfied.

where $\mathbf{a}_j^T$ denotes the jth row vector of matrix $\mathbf{A}$. Obviously, the optimal solution of the above problem can be obtained by setting the gradient of its objective function to be zero and then projecting the solution of the corresponding linear equation to region $[0, +\infty]$. Then, we can obtain

$$e_{1,j}^{k+1} = \prod_{[0,+\infty)} \frac{\mu}{\rho+\mu}\big(\varrho_j - \mathbf{a}_j^T\mathbf{v}^{k+1} - \frac{y_{1,j}^k}{\mu} + \frac{\rho}{\mu} z_{1,j}^k\big). \quad (55)$$

Similar to (46b), problem (46c) can be separated into the following N independent subproblems

$$\begin{aligned} \min_{e_{2,\ell}} \ & -\mathrm{y}_{2,\ell}^k e_{2,\ell} + \frac{\mu}{2}\left(v_\ell^k - e_{2,\ell}\right)^2 + \frac{\rho}{2}\left(e_{2,\ell} - z_{2,\ell}^k\right)^2, \\ \text{s.t.} \ & 0 \leq e_{2,\ell} \leq 1, \ \forall \ell \in \{1, \cdots, N\}. \end{aligned} \quad (56)$$

Their optimal solutions can be expressed as

$$e_{2,\ell}^{k+1} = \prod_{[0,1]} \frac{\mu}{\rho+\mu}\big(v_\ell^{k+1} + \frac{y_{2,\ell}^k}{\mu} + \frac{\rho}{\mu} z_{2,\ell}^k\big). \quad (57)$$

In *Algorithm 1*, we summarize the proposed proximal-ADMM decoding algorithm for nonbinary LDPC codes in $\mathbb{F}_{2^q}$.

V. Performance Analysis

In this section, we analyze the convergence property and computational complexity of the proposed proximal-ADMM decoding algorithm.

A. Convergence

We have the following theorem to characterize the convergence property of the proposed proximal-ADMM decoding algorithm.

Theorem 1: Assume that $\rho > \alpha > 0$ and $\frac{\alpha}{\lambda_{\min}(\mathbf{A}^T\mathbf{A})} \leq \mu \leq \frac{\rho(\rho-\alpha)^2}{4\delta_{\mathbf{AI}}^2(\rho+L+2)^2-(\rho-\alpha)^2}$, where $\delta_{\mathbf{AI}}$ is the spectral norm of matrix $\begin{bmatrix} \mathbf{A} & \mathbf{I}_M & \mathbf{0} \\ \mathbf{I}_N & \mathbf{0} & -\mathbf{I}_N \end{bmatrix}$ and L denotes the Lipschitz constant for gradient $\nabla_{\mathbf{v}}\mathcal{L}_\mu$. Let $\{\mathbf{v}^k, \mathbf{e}_1^k, \mathbf{e}_2^k, \mathbf{p}^k, \mathbf{z}_1^k, \mathbf{z}_2^k, \mathbf{y}_1^k, \mathbf{y}_2^k\}$ be the tuples generated by *Algorithm 1*. Then, we have the following convergence results

$$\begin{aligned} &\lim_{k\to+\infty} \mathbf{v}^k = \mathbf{v}^*, \ \lim_{k\to+\infty} \mathbf{e}_1^k = \mathbf{e}_1^*, \ \lim_{k\to+\infty} \mathbf{e}_2^k = \mathbf{e}_2^*, \\ &\lim_{k\to+\infty} \mathbf{p}^k = \mathbf{p}^*, \ \lim_{k\to+\infty} \mathbf{z}_1^k = \mathbf{z}_1^*, \ \lim_{k\to+\infty} \mathbf{z}_2^k = \mathbf{z}_2^*, \\ &\lim_{k\to+\infty} \mathbf{y}_1^k = \mathbf{y}_1^*, \ \lim_{k\to+\infty} \mathbf{y}_2^k = \mathbf{y}_2^*, \ \mathbf{A}\mathbf{v}^* + \mathbf{e}_1^* - \boldsymbol{\varrho} = \mathbf{0}, \\ &\mathbf{v}^* = \mathbf{e}_2^*, \quad \mathbf{v}^* = \mathbf{p}^*, \quad \mathbf{e}_1^* = \mathbf{z}_1^*, \quad \mathbf{e}_2^* = \mathbf{z}_2^*. \end{aligned} \quad (58)$$

Moreover, $\mathbf{v}^*$ is a stationary point of the original problem (43), i.e.,

$$(\mathbf{v} - \mathbf{v}^*)^T \nabla_{\mathbf{v}} g(\mathbf{v}^*) \geq 0, \quad \forall \mathbf{v} \in X, \quad (59)$$

where $g(\mathbf{v}) = \boldsymbol{\lambda}^T\mathbf{v} - \frac{\alpha}{2}\|\mathbf{v} - 0.5\|_2^2$.

Remark: The complete proof of the above convergence theorem is presented in [45] as complementary material of this paper. The above *Theorem 1* demonstrates that the proposed proximal-ADMM decoder is theoretically-guaranteed convergent to some stationary point of the non-convex decoding problem (43). Moreover, we should note that the convergence of the state-of-the-art decoders, including ADMM-based decoders and BP-like decoders, do not have this kind of convenient property. Furthermore, the value of parameter μ can be determined efficiently since $\mathbf{A}^T\mathbf{A}$ is block diagonal. In the following subsection, we show that the proposed proximal-ADMM decoder's computational complexity is also competitive.

B. Computational Complexity

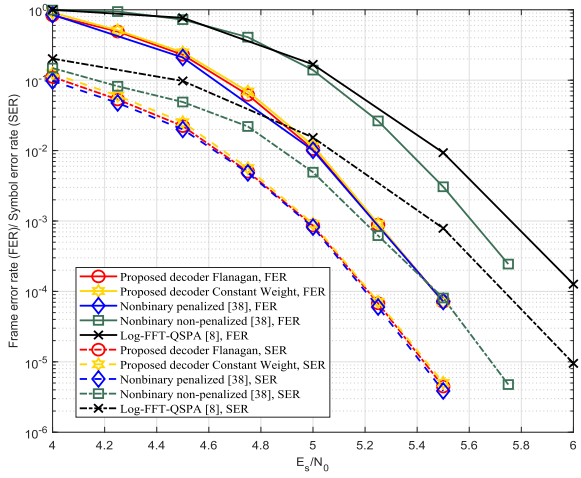
In the subsection, we show the complexity analysis of the proposed proximal-ADMM decoding algorithm in each iteration. Moreover, the presented result only includes multiplications since they occupy a dominant computation resource in practice. Before providing the analysis on the computational complexity of *Algorithm 1*, we show matrix $\mathbf{A}$ has the following property.

Fact 2: The elements in matrix $\mathbf{A}$ are 0, 1 or -1.

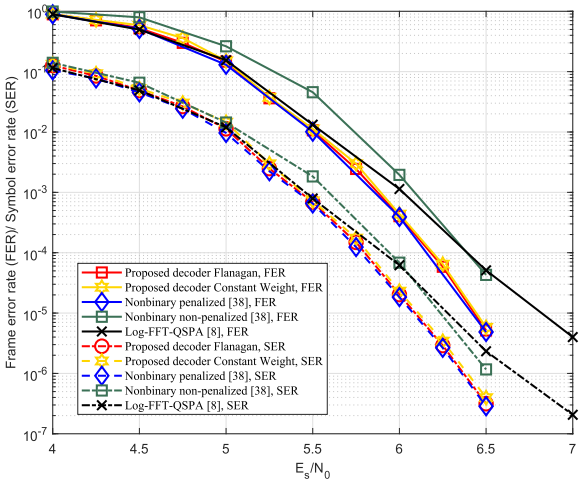
Proof: See Appendix D. ∎

Based on the above fact of matrix $\mathbf{A}$, we can see that all multiplications with regard to $\mathbf{A}$ can be replaced by additions. Moreover, $(\boldsymbol{\lambda} + 0.5\alpha)/\mu$, θ_i, ω_i and $\frac{\mu}{\rho+\mu}$ can be calculated in advance before we start the proximal-ADMM iterations.

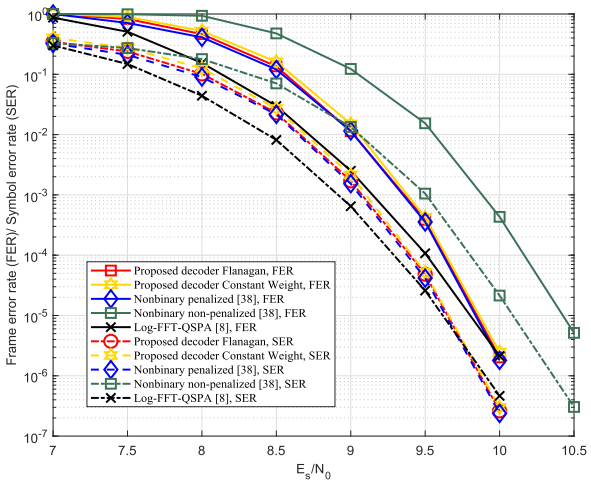
Consider $\mathbf{v}^{k+1}$ first. Calculating $\boldsymbol{\varphi}_i^k$ in (48) only requires $2^q - 1$ multiplications, which comes from computing $\frac{\rho}{\mu}\mathbf{p}_i^k$. Here, notice $\mathbf{p}_i^k$ is the (2^q-1)-length sub-vector of $\mathbf{p}^k$. Moreover, from (53), we can observe that computing $(\theta_i - \omega_i)\boldsymbol{\varphi}_i^k$ and $\omega_i \sum_{\iota=1}^{q-1} \varphi_{i,\iota}^k$ requires $2^q - 1$ multiplications and one multiplication respectively. Therefore, updating $\mathbf{v}_i^{k+1}$ requires $2^{q+1} - 1$ multiplications. This implies that the $\mathbf{v}^{k+1}$-update needs $(n+\Gamma_a)(2^{q+1}-1)$ multiplications; next, consider updating $\mathbf{e}_1^{k+1}$ and $\mathbf{e}_2^{k+1}$. From (55), we observe that each $e_{1,j}^{k+1}$ can be updated only via two multiplication operations. Thus, the multiplication number of the $\mathbf{e}_1^{k+1}$-update is 2M. Similarly, observing (57), we can find that computing $e_{2,\ell}^{k+1}$ also requires only two multiplications. As a result, it takes $2N$ multiplications to obtain $\mathbf{e}_2^{k+1}$; third, from (46d)–(46f),



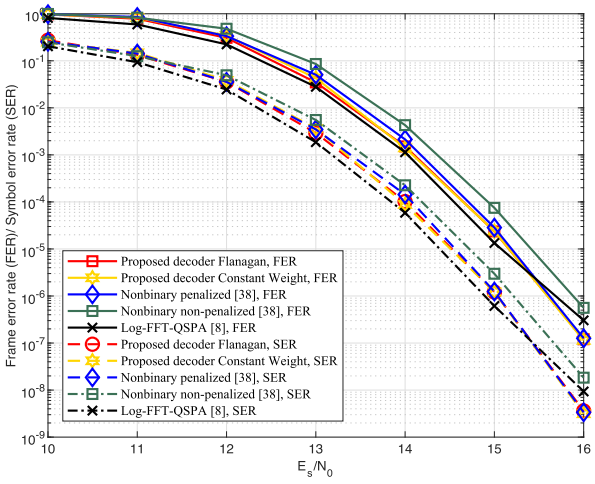
(a) Tanner (1055,424)-$\mathcal{C}_1$ in $\mathbb{F}_4$ with QPSK modulation.



(b) PEG (504,252)-$\mathcal{C}_2$ in $\mathbb{F}_4$ with QPSK modulation.



(c) MacKay (204,102)-$\mathcal{C}_3$ in $\mathbb{F}_8$ with 8PSK modulation.



(d) Tanner (155,64)-$\mathcal{C}_4$ in $\mathbb{F}_{16}$ with 16QAM.

Fig. 2. Comparisons of FER/SER performance for four LDPC codes from [40] and [41] in different Galois fields with different modulations.

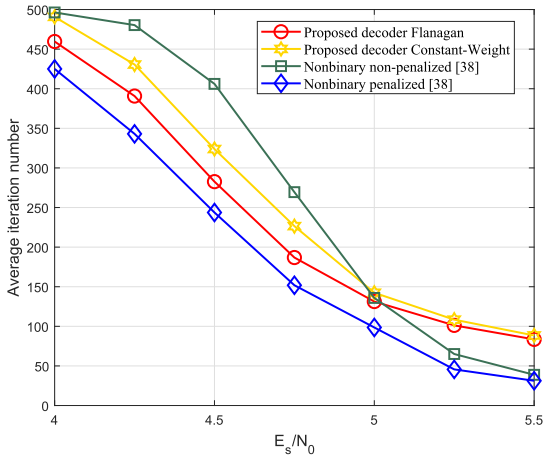
we easily observe that updating $\mathbf{p}^{k+1}$, $\mathbf{z}_1^{k+1}$, and $\mathbf{z}_2^{k+1}$ requires N, M, and N multiplications respectively. In addition, observing variables $\mathbf{y}_1$ and $\mathbf{y}_2$ in (53), (55), and (57), one can find that if their scaled forms $\frac{\mathbf{y}_1}{\mu}$ and $\frac{\mathbf{y}_2}{\mu}$ are updated in the iteration procedure, they are free of multiplications, i.e., calculating $\frac{\mathbf{y}_1^{k+1}}{\mu}$ and $\frac{\mathbf{y}_2^{k+1}}{\mu}$ only requires some addition operations. From the above analysis, one can see that the overall multiplications of *Algorithm 1* in each iteration are $(n+\Gamma_a)(2^{q+1}-1)+3N+2M$. Since $M=4(2^q-1)\Gamma_c+n+\Gamma_a$ and $N=(2^q-1)(n+\Gamma_a)$ (see (44)), it can be rewritten as

$$(6\cdot 2^q-2)(n+\Gamma_a)+12(2^q-1)\Gamma_c. \tag{60}$$

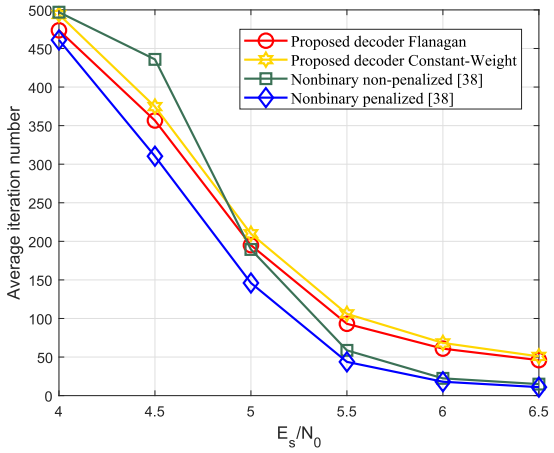
Moreover, (13) indicates $\Gamma_a \le m(d-3)=n(1-R)(d-3)$ and $\Gamma_c \le m(d-2)=n(1-R)(d-2)$, where R denotes the code rate and d is the largest check node degree. Then, one can see that either Γ_a or Γ_c is proportional to the code length n since $d \ll n$ in the case of nonbinary LDPC codes. Therefore, we can conclude that the computational complexity of the proposed proximal-ADMM decoding algorithm in every iteration scales linearly with nonbinary LDPC code length and the size of the Galois field, or roughly $\mathcal{O}(2^q n)$. It is comparable to the BP-like algorithm [4] and cheaper than ADMM decoders proposed in [38] especially when q is large. Moreover, we should note that sub-vectors $\mathbf{v}_i$ and entries in $\mathbf{e}_1$, $\mathbf{e}_2$, $\mathbf{p}$, $\mathbf{z}_1$, and $\mathbf{z}_2$ can be updated in parallel. In Table II, we summarize the above analysis on computational complexity in the decoding procedure.

VI. Simulation Results

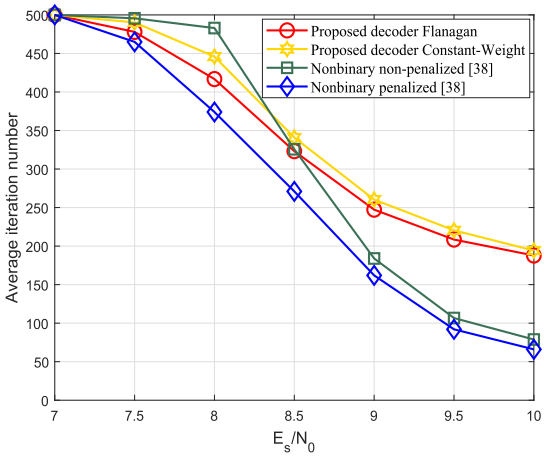
In this section, several numerical results are presented for the proposed proximal-ADMM decoder. First, we show their error-correction performance (frame error rate (FER) and symbol error rate (SER)) and decoding efficiency, which are compared with several state-of-the-art nonbinary LDPC decoders. Second, we present how to select the proper value of the parameters in the proximal-ADMM decoders, which can improve their error-correction performance and convergence rate.



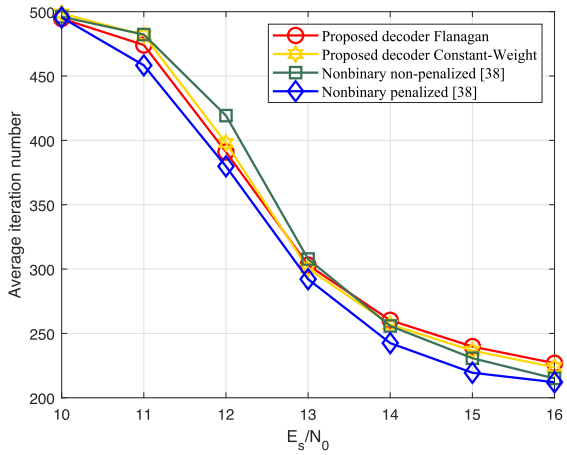
(a) Tanner (1055,424) code $\mathcal{C}_1$ in $\mathbb{F}_4$ with QPSK modulation.



(b) PEG (504,252) code $\mathcal{C}_2$ in $\mathbb{F}_4$ with QPSK modulation.



(c) MacKay (204,102) code $\mathcal{C}_3$ in $\mathbb{F}_8$ with 8PSK modulation.



(d) Tanner (155,64) code $\mathcal{C}_4$ in $\mathbb{F}_{16}$ with 16QAM modulation.

Fig. 3. Comparisons of the average number of iterations for four LDPC codes from [40] and [41] using different decoders with different modulations, where $\mathcal{C}_1$ denotes the Tanner (1055,424) code, $\mathcal{C}_2$ denotes the PEG (504,252) code, $\mathcal{C}_3$ denotes the MacKay (204,102) code, and $\mathcal{C}_4$ denotes the Tanner (155,64) code.

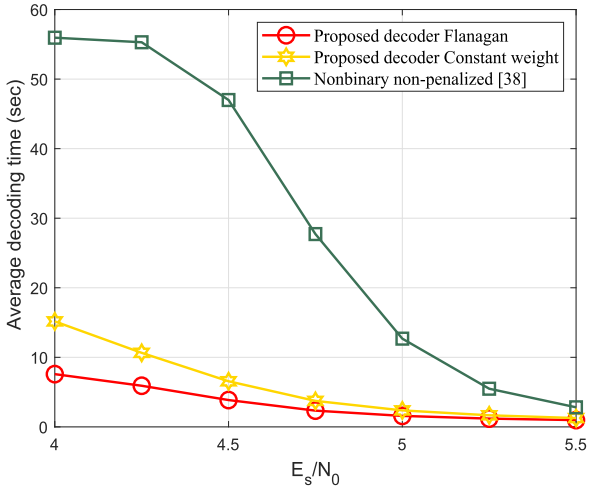
A. Comparison of Decoding Performance

We consider four LDPC codes, which are Tanner (1055,424)-$\mathcal{C}_1$ from [41], irregular PEG (504,252)-$\mathcal{C}_2$ from [40], rate-0.5 MacKay (204, 102)-$\mathcal{C}_3$ from [40], and Tanner (155,64)-$\mathcal{C}_4$ from [41] respectively. For $\mathcal{C}_1$ and $\mathcal{C}_2$, we use the same parity-check matrix as the binary case, but its entries are cast as elements in $\mathbb{F}_4$. The two codes are modulated by quadrature phase shift keying (QPSK). Similar to the approach in [32], we set the first/second/third/fourth/fifth/sixth nonzero entries in each row of the parity check matrix of $\mathcal{C}_3$ to 1/4/6/5/2/1$\in \mathbb{F}_8$ respectively. The code symbols of $\mathcal{C}_3$ are modulated by 8 phase-shift keying (8PSK). For $\mathcal{C}_4$, its parity-check matrix is the same to the binary case, but its entries are cast as elements in $\mathbb{F}_{16}$. The corresponding modulation is sixteen quadrature amplitude modulation (16QAM). The modulated symbols are transmitted over the AWGN channel. The considered decoders include the proposed two proximal-ADMM algorithms, logarithm-domain fast-fourier-transforms-based Q-ary sum-product algorithm (Log-FFT-QSPA) [4], and non-penalized/penalized ADMM decoders in [38]. The transmitted information symbols are generated randomly. The parameters of the two

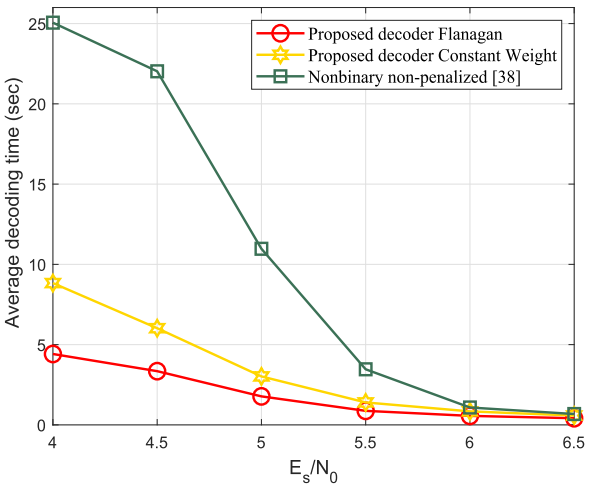
TABLE II

SUMMARY OF COMPUTATION COMPLEXITY IN EACH PROXIMAL-ADMM ITERATION

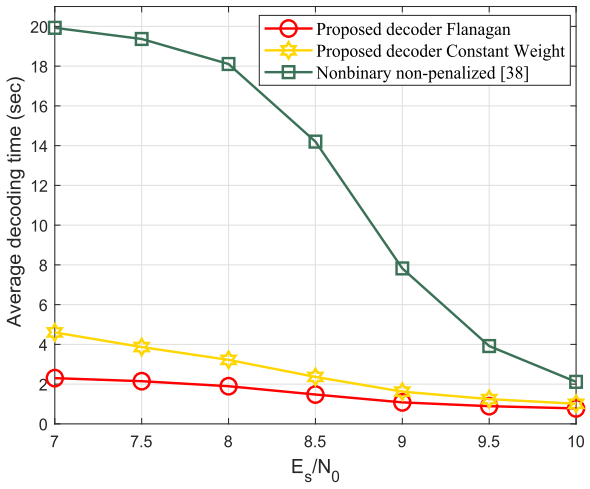
Variables	Equations	Multiplication Number
$\mathbf{v}^{k+1}$	(47)	$(2^{q+1}-1)(n+\Gamma_a)$
$\mathbf{e}_1^{k+1}$	(55)	$2(4(2^q-1)\Gamma_c+n+\Gamma_a)$
$\mathbf{e}_2^{k+1}$	(57)	$2(2^q-1)(n+\Gamma_a)$
$\mathbf{p}^{k+1}$	(46d)	$(2^q-1)(n+\Gamma_a)$
$\mathbf{z}_1^{k+1}$	(46e)	$4(2^q-1)\Gamma_c+n+\Gamma_a$
$\mathbf{z}_2^{k+1}$	(46f)	$(2^q-1)(n+\Gamma_a)$
$\mathbf{y}_1^{k+1}/\mu_1$	(46g)	free
$\mathbf{y}_2^{k+1}/\mu_2$	(46h)	free
Total		$(6\cdot 2^q-2)(n+\Gamma_a)+12(2^q-1)\Gamma_c$



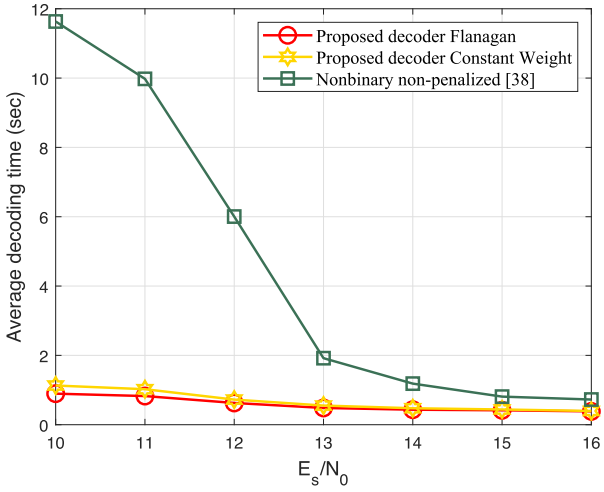
(a) Tanner (1055,424)-$\mathcal{C}_1$ in $\mathbb{F}_4$ with QPSK modulation.



(b) PEG (504,252)-$\mathcal{C}_2$ in $\mathbb{F}_4$ with QPSK modulation.



(c) MacKay (204,102)-$\mathcal{C}_3$ in $\mathbb{F}_8$ with 8PSK modulation.



(d) Tanner (155,64)-$\mathcal{C}_4$ in $\mathbb{F}_{16}$ with 16QAM modulation.

Fig. 4. Comparisons of the average decoding time of the proposed decoders and non-penalized decoder in [38], where LDPC codes are selected from [40] and [41]. Here, we should note that, at the very low SNR region, both of the decoders reach their maximum iteration number of 500. Notice that execution time of the penalized decoder in [38] is not presented here since it will show the difference unclearly among the three decoders. In [38], authors observed empirically that the penalized decoder is around 20 times slower than the non-penalized one, which was also observed in our simulations.

proximal-ADMM algorithms are set the same, where penalty parameter μ is chosen as 0.8, 0.6, 0.7, and 0.6 for codes $\mathcal{C}_1$-$\mathcal{C}_4$ respectively; parameters α, ρ, and β are set to be 0.5, 0.52, and 0.9 respectively for both of the two codes; all the input codewords are generated randomly; we stop the iteration when either $\|\mathbf{A}\mathbf{v}^k + \mathbf{e}_1^k - \boldsymbol{\varrho}\|_2^2 \leq 10^{-5}$ or $\|\mathbf{v}^k - \mathbf{e}_2^k\|_2^2 \leq 10^{-5}$ is satisfied, or the maximum iteration number $t_{\max} = 500$ is reached. All of the simulations are performed in MATLAB 2019b/Windows 7 environment on a computer with 3GHz Intel i5-9500 CPU and 8GB RAM.

Figure 2 shows FER/SER curves of code $\mathcal{C}_1 - \mathcal{C}_4$ when different decoders are applied, where all data points are based on generating at least 200 error frames, except for the last two points where 50 error frames are observed due to limited computational resources. From figures 2(a)-2(d), one can observe that the proposed decoders have similar FER/SER performance to the penalized ADMM decoder in [38], but are much better than the non-penalized one. These indicate that the penalty term plays an important role in the decoding procedure. Moreover, from figures 2(a), 2(b), and 2(d), one can see that the proposed decoders' FER/SER performance is better than Log-FFT-QSPA [6] in the high SNR region, where FER/SER curves of the proposed two decoders continue to drop in a waterfall manner while Log-FFT-QSPA decoder's decrease slowly. Specifically, in figure 2(c), the proposed decoders' FER/SER performance is inferior to the Log-FFT-QSPA decoder at considered SNRs. However, we note that the slopes of the FER/SER curves of the proposed decoders are steeper than those of the Log-FFT-QSPA decoder at high SNRs. This implies that they could surpass the Log-FFT-QSPA decoder in much higher regions, for which we did not simulate due to limited computational resources. In summary,

we see that the proposed proximal-ADMM decoders have better error-correction performance than the Log-FFT-QSPA decoder at a high SNR region, especially for long nonbinary LDPC codes. The reason could be that both of them are always trying to determine the global optimal solution of the decoding problem, but the Log-FFT-QSPA decoder is designed to search the solution locally.

Figure 3 and Figure 4 show the averaged iteration numbers and decoding time of the proposed two proximal-ADMM decoders and the non-penalized/penalized ADMM decoders in [38] respectively. In figure 3, all data points are averaged over one million LDPC frames. From them, it can be observed that the averaged number of iterations required by the ADMM decoders in [38] are less than the proposed proximal-ADMM decoders in high SNR regions for the considered codes $\mathcal{C}_1$- $\mathcal{C}_4$. The reason could be that more auxiliary variables are involved in the decoding model[9]. In figure 4, all data points of the curves are also averaged over one million LDPC frames. From the figures, one can see that the proposed two decoders cost less decoding time than the competing ADMM decoders [38] for the considered codes $\mathcal{C}_1$- $\mathcal{C}_4$. Moreover, the proximal-ADMM decoder based on the Constant-Weight embedding technique takes a little bit longer for decoding than the one based on the Flanagan embedding technique, which verifies the computational analysis in Appendix E.

B. Parameter Choices of the Proposed Proximal-ADMM Decoder

Here, we just focus on the proximal-ADMM decoder based on the Flanagan embedding technique. There are several parameters in the proposed proximal-ADMM decoding *Algorithm 1*, including the penalty parameter μ, the 2-norm penalty parameter α, the ending tolerance ξ, the maximum iteration number $t_{\max}$, and parameters ρ and β. Proper parameters can make *Algorithm 1* achieve favorable error-correction performance and reduce the iteration number. It is easy to see that a sufficiently large $t_{\max}$ and sufficiently small ξ can lead to good error-correction performance for *Algorithm 1*. Thus, we fix the ending tolerance $\xi = 10^{-5}$ and the maximum number of iterations $t_{max} = 500$ in the simulations. Moreover, a proper large β is favorable because it can update variables $\mathbf{v}$, $\mathbf{e}_1$, and $\mathbf{e}_2$ in every proximal-ADMM iteration and not deviate too much from the stabilized $\mathbf{p}$, $\mathbf{z}_1$, and $\mathbf{z}_2$ respectively (c.f. [44]). Therefore, we set β to be 0.9 in the simulations. Moreover, *Algorithm 1* is also sensitive to the values of parameters μ, α, and ρ. In order to guarantee that subproblem (46a) is strongly convex with respect to variable $\mathbf{x}$, we let $\rho > \alpha$.

[9]In comparison with the proposed proximal-ADMM decoders, non-penalized/penalized decoders in [38] need less variables/checks. Usually, this merit leads the decoder to costing less dynamic power [47] when implementing it using an FPGA chip. However, the decoding procedure involves high-dimensional Euclidean projections, which have to be implemented in series. This means that the corresponding working-frequency is higher when they desire the same decoding throughput to the proximal-ADMM decoders. From a practical viewpoint, dynamic power is an important parameter, which is related to many factors, such as variables (wires/logical resource), working frequency, voltage, etc. involved in the decoding procedure. Therefore, to evaluate dynamic power of a decoder is a complex (but important) task, which should be considered carefully in practice.

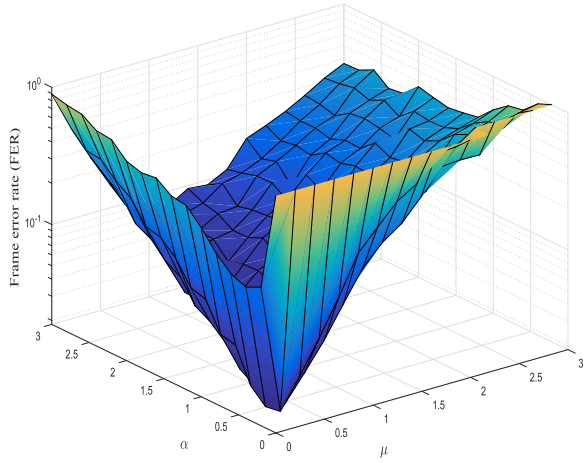


Fig. 5. FER performance plotted as a function of μ and α for the Tanner [1055,424] code $\mathcal{C}_1$ in $\mathbb{F}_4$.

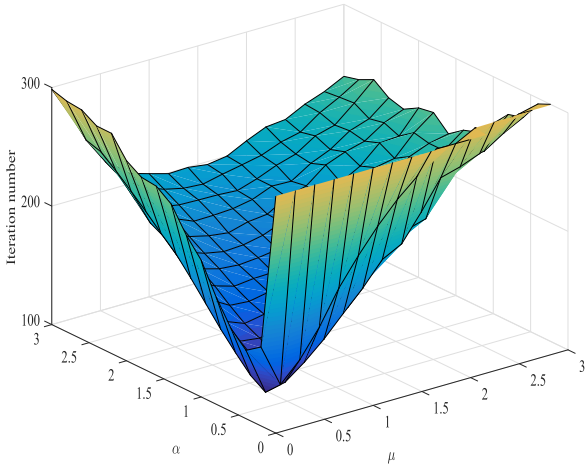


Fig. 6. Number of iterations plotted as a function of μ and α for the Tanner [1055,424] code $\mathcal{C}_1$ in $\mathbb{F}_4$.

Next, we focus on how to choose parameters μ and α. Figure 5 and figure 6 plot FER performance and iteration numbers for code $\mathcal{C}_1$ as a function of parameters μ and α at $E_s/N_0 = 5\,\text{dB}$ respectively. In the figures, we set $\rho = \alpha + 0.02$ to ensure that $\rho > \alpha$ always holds. Observing figure 5, one can find that *Algorithm 1* achieves better FER performance when $\mu \in (0.3, 1)$ and $\alpha \in (0.2, 0.5)$. Moreover, from Figure 6, one can see that the decoder takes fewer iterations when $\mu \in (0.3, 1)$ and $\alpha \in (0.2, 0.5)$. This means that $\mu \in (0.3, 1)$ and $\alpha \in (0.2, 0.5)$ are good choices in terms of error-correction performance and decoding efficiency of *Algorithm 1*.

VII. Conclusion

In this paper, two efficient decoders are developed for nonbinary LDPC codes in $\mathbb{F}_{2^q}$ via proximal-ADMM techniques based on the Flanagan/Constant-Weight embedding technique respectively. We show that both of their decoding

complexities scale linearly with block length and Galois field's size of the nonbinary LDPC codes and the corresponding iteration algorithms converge to some stationary point of the formulated decoding model. Besides, the latter one has a codeword symmetry property. Simulation results demonstrate the effectiveness of the proposed proximal-ADMM decoders in comparison with several state-of-the-art decoders.

APPENDIX A
PROOF OF FACT 1

Proof: By adding both sides of equations (10)-(12), we obtain

$$h_{\sigma_1}c_{\sigma_1}+h_{\sigma_2}c_{\sigma_2}+g_1+g_1+h_{\sigma_3}c_{\sigma_3}+g_2+g_2+\ldots+g_{d_i-3} +g_{d_i-3}+h_{\sigma_{d_j-1}}c_{\sigma_{d_j-1}}+h_{\sigma_{d_j}}c_{\sigma_{d_j}}=0, \quad (61)$$

where the sum is in $\mathbb{F}_{2^q}$. Since $g_t+g_t=0$ in $\mathbb{F}_{2^q}$, the above equation can be reduced to

$$h_{\sigma_1}c_{\sigma_1}+h_{\sigma_2}c_{\sigma_2}+h_{\sigma_3}c_{\sigma_3}+\ldots+h_{\sigma_{d_j-1}}c_{\sigma_{d_j-1}}+h_{\sigma_{d_j}}c_{\sigma_{d_j}}=0, \quad (62)$$

which is just the jth check equation in (1). It can be equivalent to

$$h_{\sigma_1}c_{\sigma_1}+h_{\sigma_2}c_{\sigma_2}+g_1=0, \quad (63a)$$
$$g_1+h_{\sigma_3}c_{\sigma_3}+\ldots+h_{\sigma_{d_j-1}}c_{\sigma_{d_j-1}}+h_{\sigma_{d_j}}c_{\sigma_{d_j}}=0. \quad (63b)$$

Moreover, (63b) can be further divided into

$$g_1+h_{\sigma_3}c_{\sigma_3}+g_2=0, \quad (64a)$$
$$g_2+\ldots+h_{\sigma_{d_j-1}}c_{\sigma_{d_j-1}}+h_{\sigma_{d_j}}c_{\sigma_{d_j}}=0. \quad (64b)$$

Through similar derivations, we obtain

$$\begin{aligned} &h_{\sigma_1}c_{\sigma_1}+h_{\sigma_2}c_{\sigma_2}+g_1=0,\\ &g_{t-1}+h_{\sigma_{t+1}}c_{\sigma_{t+1}}+g_t=0, \quad t=2,\cdots,d_j-3,\\ &g_{d_i-3}+h_{\sigma_{d_j-1}}c_{\sigma_{d_j-1}}+h_{\sigma_{d_j}}c_{\sigma_{d_j}}=0. \end{aligned} \quad (65)$$

Thus, one can conclude that any general check equation in (1) can be equivalent to the three-variables parity-check equations (10)-(12). ∎

APPENDIX B
PROOF OF LEMMA 1

Proof: Let $\mathbb{I}(\cdot)$ be the indicator function defined in $\mathbb{F}_{2^q}$, i.e., $\mathbb{I}(c,\hat{c})=1$ when $c=\hat{c}$ and $\mathbb{I}(c,\hat{c})=0$ otherwise. Then, from the mapping rule defined in (4), binary vector codeword $\mathbf{x}_k$, corresponding to $c_k\in\mathbb{F}_{2^q}$, can be expressed by

$$\mathbf{x}_k=[\mathbb{I}(1,c_k);\ldots;\mathbb{I}(j,c_k);\ldots;\mathbb{I}(2^q-1,c_k)]. \quad (66)$$

Assuming $j=c_k$, it follows that $\mathbb{I}(j,c_k)=1$ and other elements in $\mathbf{x}_k$ are zeros. Moreover, based on the definition of $D(2^q,h_k)_{ij}$ in (17), matrix $\mathbf{D}(2^q,h_k)$ can be rewritten as (67), shown at the bottom of the page. Then, we have

$$\mathbf{D}(2^q,h_k)\mathbf{x}_k=\begin{bmatrix}\mathbb{I}(1,jh_k)\mathbb{I}(j,c_k)\\ \vdots\\ \mathbb{I}(i,jh_k)\mathbb{I}(j,c_k)\\ \vdots\\ \mathbb{I}(2^q-1,jh_k)\mathbb{I}(j,c_k)\end{bmatrix}. \quad (68)$$

Letting $j=c_k$ and $i=h_kc_k$, we can derive $\mathbf{D}(2^q,h_k)\mathbf{x}_k$ as

$$\mathbf{D}(2^q,h_k)\mathbf{x}_k=\begin{bmatrix}\mathbb{I}(1,h_kc_k)\\ \vdots\\ \mathbb{I}(i,h_kc_k)\\ \vdots\\ \mathbb{I}(2^q-1,h_kc_k)\end{bmatrix}=\begin{bmatrix}0\\ \vdots\\ 1\\ \vdots\\ 0\end{bmatrix}, \quad (69)$$

which completes the proof. ∎

APPENDIX C
PROOF OF LEMMA 2

Proof: According to the definition of matrix $\mathbf{A}$ in (42a), we have

$$\begin{aligned}&\mathbf{A}^T\mathbf{A}\\ &=\Big(\sum_{\tau=1}^{\Gamma_c}\Big(\hat{\mathbf{W}}_\tau(\mathbf{Q}_\tau\otimes\mathbf{I}_{2^q-1})\Big)^T\Big(\hat{\mathbf{W}}_\tau(\mathbf{Q}_\tau\otimes\mathbf{I}_{2^q-1})\Big)\Big)+\mathbf{S}^T\mathbf{S}\\ &=\Big(\sum_{\tau=1}^{\Gamma_c}(\mathbf{Q}_\tau\otimes\mathbf{I}_{2^q-1})^T\hat{\mathbf{W}}_\tau^T\hat{\mathbf{W}}_\tau(\mathbf{Q}_\tau\otimes\mathbf{I}_{2^q-1})\Big)+\mathbf{S}^T\mathbf{S},\end{aligned} \quad (70)$$

where $\hat{\mathbf{W}}_\tau=\Big[\mathbf{P}\hat{\mathbf{T}}_1\mathbf{D}_\tau;\cdots;\mathbf{P}\hat{\mathbf{T}}_{2^q-1}\mathbf{D}_\tau\Big]$, $\hat{\mathbf{T}}_\ell=\sum_{i\in\mathcal{K}_\ell}\mathbf{T}_i$, $\ell=1,\cdots,2^q-1$, and $\mathcal{K}_1=\{1\}$, $\mathcal{K}_2=\{2\}$, $\mathcal{K}_3=\{1,2\}$, $\cdots,\mathcal{K}_{2^q-1}=\{1,\cdots,q\}$.[10]

Moreover, we have the following derivations for $\hat{\mathbf{W}}_\tau^T\hat{\mathbf{W}}_\tau$

$$\begin{aligned}\hat{\mathbf{W}}_\tau^T\hat{\mathbf{W}}_\tau&=\sum_{\ell=1}^{2^q-1}\mathbf{D}_\tau^T\hat{\mathbf{T}}_\ell^T\mathbf{P}^T\mathbf{P}\hat{\mathbf{T}}_\ell\mathbf{D}_\tau\\ &\overset{a}{=}4\mathbf{D}_\tau^T\Big(\sum_{\ell=1}^{2^q-1}\hat{\mathbf{T}}_\ell^T\hat{\mathbf{T}}_\ell\Big)\mathbf{D}_\tau,\end{aligned} \quad (71)$$

[10]*Example:* In $\mathbb{F}_8$, $\mathcal{K}_1=\{1\}$, $\mathcal{K}_2=\{2\}$, $\mathcal{K}_3=\{1,2\}$, $\mathcal{K}_4=\{3\}$, $\mathcal{K}_5=\{1,3\}$, $\mathcal{K}_6=\{2,3\}$, $\mathcal{K}_7=\{1,2,3\}$.

$$\mathbf{D}(2^q,h_k)=\begin{bmatrix}\mathbb{I}(1,h_k) & \cdots & \mathbb{I}(1,jh_k) & \cdots & \mathbb{I}(1,(2^q-1)h_k)\\ \vdots & \vdots & \vdots & \vdots & \vdots\\ \mathbb{I}(i,h_k) & \cdots & \mathbb{I}(i,jh_k) & \cdots & \mathbb{I}(i,(2^q-1)h_k)\\ \vdots & \vdots & \vdots & \vdots & \vdots\\ \mathbb{I}(2^q-1,h_k) & \cdots & \mathbb{I}(2^q-1,jh_k) & \cdots & \mathbb{I}(2^q-1,(2^q-1)h_k)\end{bmatrix}. \quad (67)$$

where the equality "$\stackrel{a}{=}$" holds since $\mathbf{P}^T\mathbf{P} = 4\mathbf{I}_4$. Since $\hat{\mathbf{T}}_\ell = \mathrm{diag}(\sum_{i\in\mathcal{K}_\ell}\hat{\mathbf{b}}_i^T, \sum_{i\in\mathcal{K}_\ell}\hat{\mathbf{b}}_i^T, \sum_{i\in\mathcal{K}_\ell}\hat{\mathbf{b}}_i^T)$, $\sum_{\ell=1}^{2^q-1}\hat{\mathbf{T}}_\ell^T\hat{\mathbf{T}}_\ell$ is also diagonal and expressed as

$$\sum_{\ell=1}^{2^q-1}\hat{\mathbf{T}}_\ell^T\hat{\mathbf{T}}_\ell = \mathrm{diag}(\mathbf{\Phi},\mathbf{\Phi},\mathbf{\Phi}), \tag{72}$$

where

$$\mathbf{\Phi} = \sum_{\ell=1}^{2^q-1}\Big(\sum_{i\in\mathcal{K}_\ell}\hat{\mathbf{b}}_i\Big)\Big(\sum_{i\in\mathcal{K}_\ell}\hat{\mathbf{b}}_i^T\Big) = \begin{bmatrix} 2^{q-1} & 2^{q-2} & \cdots & 2^{q-2} \\ 2^{q-2} & 2^{q-1} & \cdots & 2^{q-2} \\ \vdots & \vdots & \ddots & \vdots \\ 2^{q-2} & 2^{q-2} & \cdots & 2^{q-1} \end{bmatrix}, \tag{73}$$

and $\mathbf{\Phi} \in \mathbb{R}^{(2^q-1)\times(2^q-1)}$. Then, we have

$$\hat{\mathbf{W}}_\tau^T\hat{\mathbf{W}}_\tau = 4\mathbf{D}_\tau^T\mathrm{diag}(\mathbf{\Phi},\mathbf{\Phi},\mathbf{\Phi})\mathbf{D}_\tau. \tag{74}$$

Since matrices $\mathbf{D}(2^q, h_{\tau_k})$, $k = 1, 2, 3$, are elementary, we can get

$$\mathbf{D}(2^q, h_{\tau_k})^T\mathbf{\Phi}\mathbf{D}(2^q, h_{\tau_k}) = \mathbf{\Phi},\ k = 1, 2, 3, \tag{75}$$

which implies that (74) can be further simplified to

$$\hat{\mathbf{W}}_\tau^T\hat{\mathbf{W}}_\tau = 4\mathrm{diag}(\mathbf{\Phi},\mathbf{\Phi},\mathbf{\Phi}). \tag{76}$$

[11] *Example 5:* In $\mathbb{F}_4$, assume there are three variable-selecting matrices $\mathbf{Q}_\tau \in \{0,1\}^{3\times4}$. They are $\mathbf{Q}_1 = \begin{bmatrix}1&0&0&0\\0&1&0&0\\0&0&0&1\end{bmatrix}$, $\mathbf{Q}_2 = \begin{bmatrix}0&1&0&0\\0&0&1&0\\0&0&0&1\end{bmatrix}$, and $\mathbf{Q}_3 = \begin{bmatrix}1&0&0&0\\0&0&1&0\\0&0&0&1\end{bmatrix}$, respectively. Notice there are four variables involved in the check equations. The first three variables are involved in two check equations and the fourth variable is involved in three check equations, i.e., $d_1 = d_2 = d_3 = 2$, $d_4 = 3$. Moreover, since $\mathbf{S} = \mathrm{diag}(\underbrace{\mathbf{1}_{2^q-1}^T,\cdots,\mathbf{1}_{2^q-1}^T}_{n+\Gamma_a})$, we have

$$\begin{aligned}\mathbf{A}^T\mathbf{A} =& 4\Big(\sum_{\tau=1}^{3}(\mathbf{Q}_\tau\otimes\mathbf{I}_3)^T\mathrm{diag}(\mathbf{\Phi},\mathbf{\Phi},\mathbf{\Phi})(\mathbf{Q}_\tau\otimes\mathbf{I}_3)\Big)+\mathbf{S}^T\mathbf{S}\\ =&4\Bigg(\begin{bmatrix}\mathbf{I}_3&\mathbf{0}&\mathbf{0}&\mathbf{0}\\\mathbf{0}&\mathbf{I}_3&\mathbf{0}&\mathbf{0}\\\mathbf{0}&\mathbf{0}&\mathbf{0}&\mathbf{I}_3\end{bmatrix}^T\mathrm{diag}(\mathbf{\Phi},\mathbf{\Phi},\mathbf{\Phi})\begin{bmatrix}\mathbf{I}_3&\mathbf{0}&\mathbf{0}&\mathbf{0}\\\mathbf{0}&\mathbf{I}_3&\mathbf{0}&\mathbf{0}\\\mathbf{0}&\mathbf{0}&\mathbf{0}&\mathbf{I}_3\end{bmatrix}\\ &+\begin{bmatrix}\mathbf{0}&\mathbf{I}_3&\mathbf{0}&\mathbf{0}\\\mathbf{0}&\mathbf{0}&\mathbf{I}_3&\mathbf{0}\\\mathbf{0}&\mathbf{0}&\mathbf{0}&\mathbf{I}_3\end{bmatrix}^T\mathrm{diag}(\mathbf{\Phi},\mathbf{\Phi},\mathbf{\Phi})\begin{bmatrix}\mathbf{0}&\mathbf{I}_3&\mathbf{0}&\mathbf{0}\\\mathbf{0}&\mathbf{0}&\mathbf{I}_3&\mathbf{0}\\\mathbf{0}&\mathbf{0}&\mathbf{0}&\mathbf{I}_3\end{bmatrix}\\ &+\begin{bmatrix}\mathbf{I}_3&\mathbf{0}&\mathbf{0}&\mathbf{0}\\\mathbf{0}&\mathbf{0}&\mathbf{I}_3&\mathbf{0}\\\mathbf{0}&\mathbf{0}&\mathbf{0}&\mathbf{I}_3\end{bmatrix}^T\mathrm{diag}(\mathbf{\Phi},\mathbf{\Phi},\mathbf{\Phi})\begin{bmatrix}\mathbf{I}_3&\mathbf{0}&\mathbf{0}&\mathbf{0}\\\mathbf{0}&\mathbf{0}&\mathbf{I}_3&\mathbf{0}\\\mathbf{0}&\mathbf{0}&\mathbf{0}&\mathbf{I}_3\end{bmatrix}\Bigg)+\mathbf{S}^T\mathbf{S}\\ =&4\left(\mathrm{diag}(\mathbf{\Phi},\mathbf{\Phi},\mathbf{0},\mathbf{\Phi})+\mathrm{diag}(\mathbf{0},\mathbf{\Phi},\mathbf{\Phi},\mathbf{\Phi})+\mathrm{diag}(\mathbf{\Phi},\mathbf{0},\mathbf{\Phi},\mathbf{\Phi})\right)\\ &+\begin{bmatrix}\mathbf{1}_{3\times3}&\mathbf{0}&\mathbf{0}&\mathbf{0}\\\mathbf{0}&\mathbf{1}_{3\times3}&\mathbf{0}&\mathbf{0}\\\mathbf{0}&\mathbf{0}&\mathbf{1}_{3\times3}&\mathbf{0}\\\mathbf{0}&\mathbf{0}&\mathbf{0}&\mathbf{1}_{3\times3}\end{bmatrix}\\ =&\mathrm{diag}(8\mathbf{\Phi}+\mathbf{1}_{3\times3},8\mathbf{\Phi}+\mathbf{1}_{3\times3},8\mathbf{\Phi}+\mathbf{1}_{3\times3},12\mathbf{\Phi}+\mathbf{1}_{3\times3})\end{aligned}$$

i.e., $\mathbf{A}^T\mathbf{A} = \mathrm{diag}(4d_1\mathbf{\Phi}+\mathbf{1}_{3\times3}, 4d_2\mathbf{\Phi}+\mathbf{1}_{3\times3}, 4d_3\mathbf{\Phi}+\mathbf{1}_{3\times3}, 4d_4\mathbf{\Phi}+\mathbf{1}_{3\times3})$.

Plugging (76) into (70) and noticing $\mathbf{S} = \mathrm{diag}(\underbrace{\mathbf{1}_{2^q-1}^T,\cdots,\mathbf{1}_{2^q-1}^T}_{n+\Gamma_a})$, we can conclude[11]

$$\begin{aligned}&\mathbf{A}^T\mathbf{A}\\ =&4\Big(\sum_{\tau=1}^{\Gamma_c}(\mathbf{Q}_\tau\otimes\mathbf{I}_{2^q-1})^T\mathrm{diag}(\mathbf{\Phi},\mathbf{\Phi},\mathbf{\Phi})(\mathbf{Q}_\tau\otimes\mathbf{I}_{2^q-1})\Big)+\mathbf{S}^T\mathbf{S}\\ =&\mathrm{diag}(4d_1\mathbf{\Phi}+\mathbf{1}_{2^q-1}\mathbf{1}_{2^q-1}^T,\cdots,4d_{n+\Gamma_a}\mathbf{\Phi}+\mathbf{1}_{2^q-1}\mathbf{1}_{2^q-1}^T),\end{aligned} \tag{77}$$

where d_i, $i = 1,\cdots,n+\Gamma_a$, denotes the number of three-variables parity-check equations that the ith information symbol is involved in.

From (77), it is easy to see that matrix $\big(\mathbf{A}^T\mathbf{A}+\epsilon\mathbf{I}_{(n+\Gamma_a)(2^q-1)}\big)^{-1}$ is block diagonal, whose ith diagonal block sub-matrix can be expressed as

$$\mathbf{A}_i^T\mathbf{A}_i+\epsilon\mathbf{I}_{2^q-1} = 4d_i\mathbf{\Phi}+\mathbf{1}_{2^q-1}\mathbf{1}_{2^q-1}^T+\epsilon\mathbf{I}_{2^q-1}. \tag{78}$$

Plugging LHS of (78) into (77), we obtain the first part of Lemma 2, i.e.,

$$\begin{aligned}\big(\mathbf{A}^T\mathbf{A}+\epsilon\mathbf{I}_{(n+\Gamma_a)(2^q-1)}\big)^{-1} =& \mathrm{diag}\Big(\big(\mathbf{A}_1^T\mathbf{A}_1+\epsilon\mathbf{I}_{2^q-1}\big)^{-1},\ldots,\\ &\big(\mathbf{A}_{n+\Gamma_a}^T\mathbf{A}_{n+\Gamma_a}+\epsilon\mathbf{I}_{2^q-1}\big)^{-1}\Big).\end{aligned}$$

For the second part of Lemma 2, we first plug (73) into (78), which leads to

$$\begin{aligned}&\mathbf{A}_i^T\mathbf{A}_i+\epsilon\mathbf{I}_{2^q-1}\\ =&\begin{bmatrix}4d_i2^{q-1}+1+\epsilon & 4d_i2^{q-2}+1 & \cdots & 4d_i2^{q-2}+1\\ 4d_i2^{q-2}+1 & 4d_i2^{q-1}+1+\epsilon & \cdots & 4d_i2^{q-2}+1\\ \vdots & \vdots & \ddots & \vdots\\ 4d_i2^{q-2}+1 & 4d_i2^{q-2}+1 & \cdots & 4d_i2^{q-1}+1+\epsilon\end{bmatrix}.\end{aligned} \tag{79}$$

Observing (79), it is easy to see that the elements in the diagonal line of matrix $\mathbf{A}_i^T\mathbf{A}_i+\epsilon\mathbf{I}_{2^q-1}$ are the same and other elements in the off-lines are the same. So its inverse matrix can be written as [43]

$$(\mathbf{A}_i^T\mathbf{A}_i+\epsilon\mathbf{I}_{2^q-1})^{-1} = \begin{bmatrix}\theta_i&\omega_i&\cdots&\omega_i\\ \omega_i&\theta_i&\cdots&\omega_i\\ \vdots&\vdots&\vdots&\vdots\\ \omega_i&\omega_i&\cdots&\theta_i\end{bmatrix}. \tag{80}$$

Multiplying both sides of (79) and (80), we have the following equalities

$$\begin{aligned}&(4d_i2^{q-1}+1+\epsilon)\theta_i+(4d_i2^{q-2}+1)(2^q-2)\omega_i=1,\\ &(4d_i2^{q-1}+1+\epsilon)\omega_i+(4d_i2^{q-2}+1)\theta_i+(4d_i2^{q-2}+1)(2^q-3)\omega_i=0,\end{aligned} \tag{81}$$

which lead to[12]

$$\begin{aligned}\theta_i &= \omega_i+\frac{1}{2^qd_i+\epsilon},\\ \omega_i &= \frac{-2^qd_i-1}{(2^qd_i+\epsilon)\big((2^{q+1}d_i+\epsilon+1)+(2^qd_i+1)(2^q-2)\big)}.\end{aligned}$$

This completes the proof. ■

[12] Here, we should note that equation (81) is only suitable for nonbinary LDPC codes, i.e., $q \geq 2$. When $q = 1$, $\mathbf{\Phi} = [1]$ and $(\mathbf{A}_i^T\mathbf{A}_i+\epsilon\mathbf{I}) = 4d_i+1+\epsilon$ (notice no ω_i in the equation), which indicates $\theta_i = \frac{1}{4d_i+1+\epsilon}$.

TABLE III
PARAMETER COMPARISON OF TWO DIFFERENT EMBEDDING TECHNIQUES

	Flanagan embedding	Constant-Weight embedding
$\gamma_{i,\sigma}$	$\log\frac{p(r_i\|x_{i,\sigma}=0)}{p(r_i\|x_{i,\sigma}=1)}$ $i=1,\cdots,n,\ \sigma=1,\cdots,2^q-1$	$\log\frac{1}{p(r_i\|x_{i,\sigma}=1)}$ $i=1,\cdots,n,\ \sigma=0,1,\cdots,2^q-1$
Rotation matrix	$\mathbf{D}(2^q,h_k)$	$\begin{bmatrix}1 & \mathbf{0}_{2^q-1}^T\\ \mathbf{0}_{2^q-1} & \mathbf{D}(2^q,h_k)\end{bmatrix}$
$\mathbf{B}$	$\left[\tilde{\mathbf{b}}(1),\ldots,\tilde{\mathbf{b}}(2^q-1)\right]$	$\left[\tilde{\mathbf{b}}(0),\tilde{\mathbf{b}}(1),\ldots,\tilde{\mathbf{b}}(2^q-1)\right]$ $\tilde{\mathbf{b}}(0)$: q-length all-zeros vector.
$\hat{\mathbf{w}}$	$\mathbf{1}_{2^q-1}\otimes\mathbf{t}$	$\mathbf{1}_{2^q}\otimes\mathbf{t}$
$\hat{\mathbf{F}}$	$[\hat{\mathbf{W}}_1(\mathbf{Q}_1\otimes\mathbf{I}_{2^q-1});\cdots;\hat{\mathbf{W}}_{\Gamma_c}(\mathbf{Q}_{\Gamma_c}\otimes\mathbf{I}_{2^q-1})]$	$[\hat{\mathbf{W}}_1(\mathbf{Q}_1\otimes\mathbf{I}_{2^q});\cdots;\hat{\mathbf{W}}_{\Gamma_c}(\mathbf{Q}_{\Gamma_c}\otimes\mathbf{I}_{2^q})]$
$\mathbf{S}$	$\mathrm{diag}(\underbrace{\mathbf{1}_{2^q-1}^T,\cdots,\mathbf{1}_{2^q-1}^T}_{n+\Gamma_a})$	$\mathrm{diag}(\underbrace{\mathbf{1}_{2^q}^T,\cdots,\mathbf{1}_{2^q}^T}_{n+\Gamma_a})$
(40)	$\mathbf{Sv}\preceq\mathbf{1}_{n+\Gamma_a}$	$\mathbf{Sv}=\mathbf{1}_{n+\Gamma_a}$
$\mathbf{A}$	$[\hat{\mathbf{W}}_1(\mathbf{Q}_1\otimes\mathbf{I}_{2^q-1});\cdots;\hat{\mathbf{W}}_{\Gamma_c}(\mathbf{Q}_{\Gamma_c}\otimes\mathbf{I}_{2^q-1});\mathbf{S}]$	$[\hat{\mathbf{W}}_1(\mathbf{Q}_1\otimes\mathbf{I}_{2^q});\cdots;\hat{\mathbf{W}}_{\Gamma_c}(\mathbf{Q}_{\Gamma_c}\otimes\mathbf{I}_{2^q});\mathbf{S}]$
(54)	$j\in\{1,\cdots,M\}$	$j\in\{1,\cdots,4(2^q-1)\Gamma_c\}$
$\mathbf{N}$	$(2^q-1)(n+\Gamma_a)$	$2^q(n+\Gamma_a)$

APPENDIX D
PROOF OF FACT 2

Proof: To be clear, we rewrite $\mathbf{A}$ in (42a) as follows:

$$\mathbf{A}=[\hat{\mathbf{W}}_1(\mathbf{Q}_1\otimes\mathbf{I}_{2^q-1});\cdots;\hat{\mathbf{W}}_\tau(\mathbf{Q}_\tau\otimes\mathbf{I}_{2^q-1});\cdots;\hat{\mathbf{W}}_{\Gamma_c}(\mathbf{Q}_{\Gamma_c}\otimes\mathbf{I}_{2^q-1});\mathbf{S}],$$

where $\hat{\mathbf{W}}_\tau=\left[\mathbf{P}\hat{\mathbf{T}}_1\mathbf{D}_\tau;\cdots;\mathbf{P}\hat{\mathbf{T}}_{2^q-1}\mathbf{D}_\tau\right]$ and $\hat{\mathbf{T}}_\ell=\mathrm{diag}(\sum_{i\in\mathcal{K}_\ell}\hat{\mathbf{b}}_i^T,\sum_{i\in\mathcal{K}_\ell}\hat{\mathbf{b}}_i^T,\sum_{i\in\mathcal{K}_\ell}\hat{\mathbf{b}}_i^T)$, $\ell=1,\cdots,2^q-1$. Here, we should note that $\hat{\mathbf{T}}_i$ is binary since the sum in $\sum_{i\in\mathcal{K}_i}\hat{\mathbf{b}}_i^T$ is in $\mathbb{F}_2$.

Since $\mathbf{D}(2^q,h_{\tau_j})$ is elementary, there is at most one nonzero element 1 in each column of $\mathbf{T}_\ell\mathbf{D}(2^q,h_{\tau_j})$. Moreover, since elements in matrix $\mathbf{P}$ are 1 or -1, elements in $\mathbf{P}\left(\sum_{i\in\mathcal{K}_\ell}\hat{\mathbf{T}}_i\right)\mathbf{D}_\tau$ are 0, 1, or -1. Therefore, we can conclude that elements in $\hat{\mathbf{W}}_\tau$ are also either 0, 1, or -1. Furthermore, since variable-selecting matrix $\mathbf{Q}_\tau$ has one nonzero element "1" at most in its each row/column, $\mathbf{Q}_\tau\otimes\mathbf{I}_{2^q-1}$ should have the same property. Therefore, it is obvious that elements in $\hat{\mathbf{W}}_\tau(\mathbf{Q}_\tau\otimes\mathbf{I}_{2^q-1})$ are also 1, -1, or 0. Besides, since matrix $\mathbf{S}$ only includes one nonzero element "1", we can conclude that matrix $\mathbf{A}$ consists of elements 1, -1, and 0. ∎

APPENDIX E
BRIEF PRESENTATION ON THE DESIGN OF THE LDPC DECODER VIA THE CONSTANT-WEIGHT EMBEDDING TECHNIQUE

In the beginning, we should note that the formulation procedure of the decoding model via the Constant-Weight embedding technique is almost the same as the Flanagan one. A few differences are shown in Table III. To facilitate understanding them, we use the same notations for the referred parameters.

Moreover, we consider the implementation of the proximal-ADMM solving algorithm when the Constant-Weight embedding technique is also almost the same as the Flaganan one. Besides the different dimensions of the vectors and matrices (2^q-1 to 2^q), the only difference is how to update variable $\mathbf{v}$. To be clear, we rewrite (47) as follows

$$\mathbf{v}^{k+1}=\left(\mathbf{A}^T\mathbf{A}+\epsilon\mathbf{I}_N\right)^{-1}\boldsymbol{\varphi}^k. \tag{82}$$

Similar to (49), we have

$$\left(\mathbf{A}^T\mathbf{A}+\epsilon\mathbf{I}_N\right)^{-1}=\mathrm{diag}\Big(\left(\mathbf{A}_1^T\mathbf{A}_1+\epsilon\mathbf{I}_{2^q}\right)^{-1},\ldots,\left(\mathbf{A}_{n+\Gamma_a}^T\mathbf{A}_{n+\Gamma_a}+\epsilon\mathbf{I}_{2^q}\right)^{-1}\Big),$$

where

$$\left(\mathbf{A}_i^T\mathbf{A}_i+\epsilon\mathbf{I}_{2^q}\right)=\begin{bmatrix}1+\epsilon & 1 & 1 & \cdots & 1\\ 1 & 4d_i2^{q-1}+1+\epsilon & 4d_i2^{q-2}+1 & \cdots & 4d_i2^{q-2}+1\\ 1 & 4d_i2^{q-2}+1 & 4d_i2^{q-1}+1+\epsilon & \cdots & 4d_i2^{q-2}+1\\ \vdots & \vdots & \vdots & \ddots & \vdots\\ 1 & 4d_i2^{q-2}+1 & 4d_i2^{q-2}+1 & \cdots & 4d_i2^{q-1}+1+\epsilon\end{bmatrix}.$$

Since its inverse has the following special structure

$$\left(\mathbf{A}_i^T\mathbf{A}_i+\epsilon\mathbf{I}_{2^q}\right)^{-1}=\begin{bmatrix}\varsigma_i & \kappa_i & \kappa_i & \cdots & \kappa_i\\ \kappa_i & \theta_i & \omega_i & \cdots & \omega_i\\ \vdots & \omega_i & \theta_i & \cdots & \omega_i\\ \vdots & \vdots & \vdots & \vdots & \vdots\\ \kappa_i & \omega_i & \omega_i & \cdots & \theta_i\end{bmatrix}, \tag{83}$$

we can obtain ς_i, κ_i, ω_i, and θ_i by solving a four-variables linear equation (the derivations are quite similar to the ones

presented in Appendix C), which are

$$\kappa_i = \frac{1}{2^q - 1 + (1+\epsilon)(\epsilon - 1 + 2^q(2^q d_i + 1))},$$
$$\varsigma_i = \frac{1 - (2^q - 1)\kappa_i}{1+\epsilon},$$
$$\theta_i = \omega_i + \frac{1}{2^q d_i + \epsilon},$$
$$\omega_i = \frac{(2^q d_i + 1)(1+\epsilon) - 1}{(2^q d_i + \epsilon)\,(2^q - 1 + (1+\epsilon)(\epsilon - 1 + 2^q(2^q d_i + 1)))}.$$

According to (83), since $(\mathbf{A}_i^T \mathbf{A}_i + \epsilon \mathbf{I}_{2^q})^{-1}$ can be expressed as

$$\mathbf{v}_k^{k+1} = \begin{bmatrix} \omega_i & \cdots & \cdots & \omega_i \\ \omega_i & \omega_i & \cdots & \omega_i \\ \vdots & \cdots & \ddots & \vdots \\ \omega_i & \omega_i & \cdots & \omega_i \end{bmatrix} + \begin{bmatrix} \varsigma_i - \kappa_i & \cdots & \cdots & 0 \\ 0 & \theta_i - \omega_i & \cdots & 0 \\ \vdots & \cdots & \ddots & \vdots \\ 0 & \cdots & \cdots & \theta_i - \omega_i \end{bmatrix} + \begin{bmatrix} \kappa_i - \omega_i & \kappa_i - \omega_i & \cdots & \kappa_i - \omega_i \\ \kappa_i - \omega_i & 0 & \cdots & 0 \\ \vdots & \cdots & \ddots & \vdots \\ \kappa_i - \omega_i & 0 & \cdots & 0 \end{bmatrix},$$

(82) can be written as

$$\mathbf{v}_k^{k+1} = (\omega_i \sum_{\iota=1}^{2^q} \varphi_{i,\iota}^k)\mathbf{1}_{2^q} + \begin{bmatrix} \varsigma_i - \kappa_i \\ \theta_i - \omega_i \\ \vdots \\ \theta_i - \omega_i \end{bmatrix} \circ \boldsymbol{\varphi}_i^k + (\kappa_i - \omega_i) \begin{bmatrix} \sum_{\iota=1}^{2^q} \varphi_{i,\iota}^k \\ \varphi_{i,1}^k \\ \vdots \\ \varphi_{i,1}^k \end{bmatrix} \quad (84)$$

where operator "$\circ$" denotes the Hardmard product. Using (84) to take the place of (53) in Algorithm 1, we can obtain the complete proximal-ADMM decoding algorithm based on the Constant-Weight embedding technique.

Furthermore, we consider the performance of the presented decoding algorithm above. The decoder's convergence property can also be characterized by *Theorem 1*. The computational complexity in each ADMM iteration is still scaled linearly with nonbinary LDPC code length and the size of the Galois field. However, we should say that its decoding complexity is slightly larger than the one using the Flanagan embedding technique since the size of the matrices and vectors involved is scaled linearly in terms of 2^q. To be clear, we show the number of multiplications used in the implementation in Table IV. Besides the guaranteed convergence and similar computational complexity, the proposed decoder based on Constant-Weight embedding satisfies a favorable property of the *all-zeros assumption*, which is described as follows.

Theorem 2: assume that the noisy channel is symmetrical. Then, the probability that the decoding algorithm based on the Constant-Weight embedding technique fails is independent of the transmitted codeword.

The proof of the above theorem is not provided in this paper. The interested readers could look up [45] for the details. Here, we just provide a short comment on it. This property is also called codeword symmetry, which is very favorable in either practice or theory since it guarantees that all of the codewords have the same error probability when they are transmitted through the AWGN channel. Theorem 2 holds since all of the nonbinary symbols in $\mathrm{GF}(2^q)$ are mapped in the same way. Moreover, since the Flanagan embedding technique treats symbol 0 differently, the corresponding decoder does not satisfy the property of codeword symmetry. However, we should note that the presented simulation results show that both of the two proximal-ADMM decoders have almost the same error-correction performance.

TABLE IV

SUMMARY OF COMPUTATION COMPLEXITY IN EACH PROXIMAL-ADMM ITERATION USING THE CONSTANT-WEIGHT EMBEDDING TECHNIQUE

Variables	Equations	Multiplications Number
$\mathbf{v}^{k+1}$	(82)	$3 \cdot 2^q(n + \Gamma_a)$
$\mathbf{e}_1^{k+1}$	(55)	$2(4 \cdot 2^q \Gamma_c + n + \Gamma_a)$
$\mathbf{e}_2^{k+1}$	(57)	$2^{q+1}(n + \Gamma_a)$
$\mathbf{p}^{k+1}$	(46d)	$2^q(n + \Gamma_a)$
$\mathbf{z}_1^{k+1}$	(46e)	$4 \cdot 2^q \Gamma_c + n + \Gamma_a$
$\mathbf{z}_2^{k+1}$	(46f)	$2^q(n + \Gamma_a)$
$\mathbf{y}_1^{k+1}/\mu_1$	(46g)	free
$\mathbf{y}_2^{k+1}/\mu_2$	(46h)	free
Total		$(7 \cdot 2^q + 3)(n + \Gamma_a) + 12 \cdot 2^q \Gamma_c$

ACKNOWLEDGMENT

The authors would like to thank Dr. Andrew Thangaraj, associate editor of this paper, and the anonymous reviewers who helped the authors to elevate the quality of this paper. They also would like to thank Prof. Zhiquan Luo from The Chinese University of Hong Kong (Shenzhen) and Dr. Zhiguo Wang from Sichuan University for their insightful discussion on proximal-ADMM technique.

REFERENCES

[1] M. C. Davey and D. MacKay, "Low-density parity check codes over GF(q)," *IEEE Commun. Lett.*, vol. 2, no. 6, pp. 165–167, Jun. 1998.

[2] I. B. Djordjevic and B. Vasic, "Nonbinary LDPC codes for optical communication systems," *IEEE Photon. Technol. Lett.*, vol. 17, no. 10, pp. 2224–2226, Oct. 26, 2005.

[3] R. Peng and R. Chen, "Design of nonbinary quasi-cyclic LDPC cycle codes," in *Proc. IEEE Inf. Theory Workshop*, Tahoe City, CA, USA, Sep. 2007, pp. 13–18.

[4] S. Hongzin and J. R. Cruz, "Reduced-complexity decoding of Q-ary LDPC codes for magnetic recording," *IEEE Trans. Magn.*, vol. 39, no. 2, pp. 1081–1087, Mar. 2003.

[5] B. Rong, T. Jiang, X. Li, and M. R. Soleymani, "Combine LDPC codes over GF(q) with Q-ary modulations for bandwidth efficient transmission," *IEEE Trans. Broadcast.*, vol. 54, no. 1, pp. 78–84, Mar. 2008.

[6] D. Declercq and M. Fossorier, "Decoding algorithms for nonbinary LDPC codes over GF(q)," *IEEE Trans. Commun.*, vol. 55, no. 4, pp. 633–643, Apr. 2007.

[7] F. R. Kschischang, B. J. Frey, and H.-A. Loeliger, "Factor graphs and the sum-product algorithm," *IEEE Trans. Inf. Theory*, vol. 47, no. 2, pp. 498–519, Feb. 2001.

[8] D. P. Bertsekas, *Nonlinear Programming*. Belmont, MA, USA: Athena Scientific, 2016.

[9] M. F. Flanagan, V. Skachek, E. Byrne, and M. Greferath, "Linear-programming decoding of nonbinary linear codes," *IEEE Trans. Inf. Theory*, vol. 55, no. 9, pp. 4134–4154, Sep. 2009.
[10] J. Bai, Y. Wang, and Q. Shi, "Efficient QP-ADMM decoder for binary LDPC codes and its performance analysis," *IEEE Trans. Signal Process.*, vol. 68, pp. 503–518, 2020.
[11] J. Feldman, M. Wainwright, and D. Karger, "Using linear progamming to decoding binary linear codes," *IEEE Trans. Inf. Theory*, vol. 51, no. 1, pp. 954–972, Jan. 2005.
[12] J. Feldman, T. Malkin, R. A. Servedio, C. Stein, and M. J. Wainwright, "LP decoding corrects a constant fraction of errors," *IEEE Trans. Inf. Theory*, vol. 53, no. 1, pp. 82–89, Jan. 2007.
[13] C. Daskalakis, A. G. Dimakis, R. M. Karp, and M. J. Wainwright, "Probabilistic analysis of linear programming decoding," *IEEE Trans. Inf. Theory*, vol. 54, no. 8, pp. 3565–3578, Aug. 2008.
[14] S. Arora, C. Daskalakis, and D. Steurer, "Message-passing algorithms and improved LP decoding," *IEEE Trans. Inf. Theory*, vol. 58, no. 12, pp. 7260–7271, Dec. 2012.
[15] T. Wadayama, "Interior point decoding for linear vector channels based on convex optimization," *IEEE Trans. Inf. Theory*, vol. 56, no. 10, pp. 4905–4921, Oct. 2010.
[16] H. Liu, W. Qu, B. Liu, and J. Chen, "On the decomposition method for linear programming decoding of LDPC codes," *IEEE Trans. Commun.*, vol. 58, no. 12, pp. 3448–3458, Dec. 2010.
[17] S. Barman, X. Liu, S. C. Draper, and B. Recht, "Decomposition methods for large scale LP decoding," *IEEE Trans. Inf. Theory*, vol. 59, no. 12, pp. 7870–7886, Dec. 2013.
[18] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Found. Trends Mach. Learn.*, vol. 3, no. 1, pp. 1–122, Jul. 2011.
[19] X. Zhang and P. H. Siegel, "Efficient iterative LP decoding of LDPC codes with alternating direction method of multipliers," in *Proc. IEEE ISIT*, Jul. 2013, pp. 1501–1505.
[20] G. Zhang, R. Heusdens, and W. B. Kleijn, "Large scale LP decoding with low complexity," *IEEE Commun. Lett.*, vol. 17, no. 11, pp. 2152–2155, Nov. 2013.
[21] X. Jiao, Y.-C. He, and J. Mu, "Memory-reduced look-up tables for efficient ADMM decoding of LDPC codes," *IEEE Signal Process. Lett.*, vol. 25, no. 1, pp. 110–114, Jan. 2018.
[22] H. Wei and A. H. Banihashemi, "An iterative check polytope projection algorithm for ADMM-based LP decoding of LDPC codes," *IEEE Commun. Lett.*, vol. 22, no. 1, pp. 29–32, Jan. 2018.
[23] H. Wei, X. Jiao, and J. Mu, "Reduced-complexity linear programming decoding based on ADMM for LDPC codes," *IEEE Commun. Lett.*, vol. 19, no. 6, pp. 909–912, Jun. 2015.
[24] J. Bai, Y. Wang, and F. C. M. Lau, "Minimum-polytope-based linear programming decoder for LDPC codes via ADMM approach," *IEEE Wireless Commun. Lett.*, vol. 8, no. 4, pp. 1032–1035, Aug. 2019.
[25] M. H. N. Taghavi and P. H. Siegel, "Adaptive methods for linear programming decoding," *IEEE Trans. Inf. Theory*, vol. 54, no. 12, pp. 5396–5410, Dec. 2008.
[26] M. Miwa, T. Wadayama, and I. Takumi, "A cutting-planemethod based on redundant rows for improving fractional distance," *IEEE J. Sel. Areas Commun.*, vol. 27, no. 6, pp. 1012–1105, Aug. 2009.
[27] A. Tanatmis, S. Ruzika, H. W. Hamacher, M. Punekar, F. Kienle, and N. Wehn, "A separation algorithm for improved LP-decoding of linear block codes," *IEEE Trans. Inf. Theory*, vol. 56, no. 7, pp. 3277–3289, Jul. 2010.
[28] X. Zhang and P. H. Siegel, "Adaptive cut generation algorithm for improved linear programming decoding of binary linear codes," *IEEE Trans. Inf. Theory*, vol. 58, no. 10, pp. 6581–6594, Oct. 2012.
[29] E. Rosnes and M. Helmling, "Adaptive linear programming decoding of nonbinary linear codes over prime fields," *IEEE Trans. Inf. Theory*, vol. 66, no. 3, pp. 4905–4921, Mar. 2020.
[30] X. Liu and S. C. Draper, "The ADMM penalized decoder for LDPC codes," *IEEE Trans. Inf. Theory*, vol. 62, no. 6, pp. 2966–2984, Jun. 2016.
[31] D. Goldin and D. Burshtein, "Iterative linear programming decoding of non-binary linear codes with linear complexity," *IEEE Trans. Inf. Theory*, vol. 59, no. 1, pp. 282–300, Jan. 2013.
[32] M. Punekar, P. O. Vontobel, and M. F. Flanagan, "Low-complexity LP decoding of nonbinary linear codes," *IEEE Trans. Commun.*, vol. 61, no. 8, pp. 3073–3085, Aug. 2013.
[33] D. Burshtein, "Iterative approximate linear programming decoding of LDPC codes with linear complexity," *IEEE Trans. Inf. Theory*, vol. 55, no. 11, pp. 4835–4859, Nov. 2009.
[34] P. Vontobel and R. Koetter, "Towards low-complexity linear-programming decoding," in *Proc. Int. Symp. Turbo Codes Rel. Topics*, Munich, Germany, Apr. 2006, pp. 1–9.
[35] M. Punekar and M. F. Flanagan, "Trellis-based check node processing for low-complexity nonbinary LP decoding," in *Proc. IEEE Int. Symp. Inf. Theory*, Jul. 2011, pp. 1653–1657.
[36] P. O. Vontobel and R. Koetter, "On low-complexity linear-programming decoding of LDPC codes," *Eur. Trans. Telecommun.*, vol. 18, no. 5, pp. 509–517, Aug. 2007.
[37] J. Honda and H. Yamamoto, "Fast linear-programming decoding of LDPC codes over GF(2^m)," *Proc. Int. Symp. Inf. Theory Appl. (ISITA)*, Honolulu, HI, USA, pp. 754–758, Oct. 2012.
[38] X. Liu and S. C. Draper, "ADMM LP decoding of non-binary LDPC codes in $\mathbb{F}_2$m," *IEEE Trans. Inf. Theory*, vol. 62, no. 6, pp. 2985–3010, Jun. 2016.
[39] J. Honda and H. Yamamoto, "Fast linear-programming decoding of LDPC codes over GF(2^m)," in *Proc. Int. Symp. Inf. Thoery Appl. (ISITA)*, Honolulu, HI, USA, Oct. 2012, pp. 754–758.
[40] D. MacKay. *Encyclopedia of Sparse Graph Codes*. [OnLine]. Available: http://www.inference.org.uk/mackay/codes/data.html
[41] R. M. Tanner, T. D. Sridhara, and T. Fuja, "A class of group-structured LDPC codes," in *Proc. Int. Symp. Commun. Theory Appl.*, Ambleside, U.K., Jul. 2001, pp. 365–370.
[42] W. Ryan and S. Lin, *Channel Coding: Classical and Modern*. Cambridge, U.K.: Cambridge Univ. Press, 2009.
[43] R. Horn, R. Horn, and C. Johnson, *Matrix Analysis*. Cambridge, U.K.: Cambridge Univ. Press, 1990.
[44] J. Zhang and Z.-Q. Luo, "A proximal alternating direction method of multiplier for linearly constrained nonconvex minimization," *SIAM J. Optim.*, vol. 30, no. 3, pp. 2272–2302, Jan. 2020.
[45] Y. Wang and J. Bai. (Dec. 2021). *Decoding Nonbinary LDPC Codes Via Proximal-ADMM Approach*. (Proofs for Convergence and Symmetrical Property are Included). [Online]. Available: https://arxiv.org/
[46] J.-S. Pang, "A posteriori error bounds for the linearly-constrained variational inequality problem," *Math. Oper. Res.*, vol. 12, no. 3, pp. 474–484, Aug. 1987.
[47] M. Wasson *et al.*, "Hardware-based linear program decoding with the alternating direction method of multipliers," *IEEE Trans. Signal Process.*, vol. 67, no. 19, pp. 4976–4990, Oct. 2019.

Yongchao Wang (Senior Member, IEEE) received the B.E. degree in communication engineering and the M.E. and Ph.D. degrees in information and communication engineering from Xidian University, Xi'an, China, in 1998, 2004, and 2006, respectively. From September 2008 to January 2010, he was a one-year Visiting Scholar and then a Post-Doctoral Researcher with the Department of Electrical and Computer Engineering, University of Minnesota, USA. From March 2016 to March 2017, he was a Visiting Scholar with Purdue University, USA. Since 2012, he has been a Professor with the Key State Laboratory of Integrated Services Network, Xidian University. To date, he has published more than 40 peer-reviewed papers as the first author or the corresponding author and issued about 20 national patents. His research interests mainly lie in the areas of signal processing for communications, machine learning algorithms, and their applications. His research works have been applied to several real telecommunication systems. Moreover, he was a recipient of several research awards, such as the Prize in Progress of Science and Technology from the Ministry of Education of the People's Republic of China, Shaanxi Province Government of China, and Xidian University.

Jing Bai received the B.E. degree in electronic information engineering from the Shaanxi University of Science and Technology, Xi'an, China, in 2014, and the Ph.D. degree in communication and information systems with Xidian University, Xi'an, in 2020. She is currently with the School of Information Science and Technology, Shijiazhuang Tiedao University. Her research interests include information theory, coding theory, and algorithm design and analysis via convex optimization in LDPC decoding.